

Cartridge & Cloud — Final Handoff and Project Operations Manual

Manual maestro de transferencia, retoma, incorporación, operación, cierre y continuidad del proyecto

VRM Games / Blas Luis Rocha González

2026-07-06

34 — Final Handoff and Project Operations Manual

Proyecto: Cartridge & Cloud

Estudio / titular del proyecto: VRM Games / Blas Luis Rocha González

Repositorio histórico esperado: BLRochaGonzalez93/U012_CartridgeAndCloud

Rama estable esperada: main

Plataforma inicial: PC / Steam, Windows x64

Versión de aplicación observada: 0.0.21

Schema integrado de persistencia de referencia: 2

Baseline técnica inspeccionada: Unity 6000.3.18f1 (5eb53e4c07), URP 17.3.0

Estado de producción de referencia: Sprints 0–15 CLOSED / PASS; Sprint 16 COMPLETED / PASS; Sprint 17 PENDING / READY TO OPEN; H6 BLOCKED / NOT RUN

Pruebas automatizadas históricamente documentadas en la working copy validada: 1215 EditMode + 70 PlayMode = 1285 PASS

Estado de publicación: Steamworks no debe considerarse operativo; no existe autorización para distribución pública, campaña comercial o claim de release

Clasificación: documento interno de gobierno, incorporación, operación, continuidad y transferencia

Naturaleza: manual de ejecución; no sustituye los contratos especializados ni convierte planes pendientes en capacidades verificadas

Estado de esta versión: actualización dirigida posterior a RC1; apta para generar el manifiesto 32 candidato y ejecutar la auditoría 16 candidata, pero no constituye todavía la baseline final inmutable

Regla principal: una persona debe poder retomar o recibir el proyecto sin conocer el historial del chat, pero nunca debe hacerlo igno-

rando la baseline, el repositorio, los registros de ejecución o la evidencia. Este manual indica cómo orientarse y operar; la autoridad técnica y de producto sigue residiendo en los documentos especializados, el código/configuración observados y los artefactos identificados.

Regla de verdad: el último mensaje, una captura, un handoff antiguo o una build local no son por sí solos la fuente de verdad. Antes de actuar se verifica: baseline documental, rama y SHA reales, working copy, versión de Unity, paquetes, escena objetivo, Build Profile, estado de tests, defectos, riesgos y evidencia disponible.

Regla de no regresión: ningún trabajo de arte, UX, optimización, refactor, publicación o documentación puede romper sistemas cerrados, invariantes de datos, persistencia, accesibilidad, licencias, seguridad o trazabilidad. Un cambio que no pueda explicarse, reproducirse, probarse y revertirse no está listo para integrarse.

Regla de continuidad: chats, memoria personal, rutas locales y credenciales individuales son ayudas temporales, no dependencias aceptables. Toda decisión, estado, procedimiento, evidencia y acceso crítico debe quedar transferible mediante repositorio, baseline, registros controlados y mecanismos de recuperación.

0. Propósito, alcance, autoridad y forma de uso

0.1. Propósito

Este manual completa el documento operativo 34 dentro del conjunto 00–34 y funciona como punto de entrada y salida. Esta versión es una actualización dirigida posterior a RC1: permite continuar la consolidación, pero no cierra por sí sola la baseline final. Consolida las instrucciones necesarias para:

1. retomar el proyecto después de una interrupción, cambio de equipo, pérdida de contexto o migración de conversación;
2. transferir el proyecto a otra persona o incorporar colaboradores sin depender del conocimiento tácito del propietario;
3. instalar y validar el entorno de desarrollo de forma reproducible;
4. identificar la fuente de verdad adecuada antes de diseñar, programar, producir arte, probar, construir o publicar;
5. operar sprints, gates, builds, QA, documentación, riesgos, seguridad, backups y comunicación;
6. preservar las decisiones históricas útiles sin tratarlas como estado actual cuando han sido sustituidas;
7. ejecutar un handoff completo con código, documentación, accesos, riesgos, tareas y evidencia;

8. reducir el punto único de fallo asociado a una sola persona, una sola máquina, una sola cuenta o un solo chat;
9. definir rutinas diarias, semanales, de sprint, de hito, de release y de incidente;
10. proporcionar checklists y plantillas reutilizables para nuevas sesiones, colaboradores, builds y transferencias;
11. impedir cierres falsos basados únicamente en tests verdes, compilación o aprobación informal;
12. preparar el manifiesto 32 candidato, la auditoría 16 candidata, las correcciones, la auditoría final y el paquete inmutable.

0.2. Qué resuelve y qué no resuelve

El manual resuelve la orientación y la operación transversal. No sustituye:

- 00 para la identidad y límites del producto;
- 01 para el diseño de juego;
- 02 para el contrato de H6 y Golden Path;
- 03 y 04 para arquitectura, datos e invariantes;
- 05 y 19 para UX/UI;
- 06, 12 y 13 para planificación y cambio;
- 07, 08 y 31 para QA y gates;
- 10 y 11 para setup y builds;
- 17–30 para disciplinas especializadas;
- 32 para el inventario e integridad documental;
- 33 para riesgos, BIA, continuidad y recuperación.

Cuando este manual resume una regla, el documento especializado conserva la definición completa. Cuando el manual añade un procedimiento de coordinación, ese procedimiento es normativo mientras no contradiga una autoridad superior.

0.3. Audiencias

Audiencia	Uso principal
Propietario / director del proyecto	decidir prioridades, accesos, aceptación, riesgos y transferencia
Producción	mantener estado, dependencias, sprints, gates y cierres
Ingeniería	preparar entorno, implementar, validar, versionar y recuperar
Diseño	interpretar alcance, reglas, contenido, balance y criterios
Arte / audio / UI	producir e integrar assets sin romper contratos técnicos o legales

Audiencia	Uso principal
QA	preparar builds, ejecutar pruebas, conservar evidencia y bloquear cierres inválidos
Publishing / marketing / comunidad	comunicar solo claims autorizados y operar canales cuando se abran
Legal / privacidad / seguridad	validar derechos, datos, cuentas, incidentes y distribución
Colaborador temporal	completar una tarea acotada con mínimo privilegio y handoff de salida
Receptor del proyecto	reconstruir contexto, accesos, entorno, estado y próximo trabajo

0.4. Modos de lectura

Retoma rápida

Leer, en este orden:

1. secciones 1, 2 y 3 de este manual;
2. sección 8, «Estado operativo y próxima secuencia»;
3. 31_H6_and_Release_Readiness_Checklist.xlsx, dashboard y controles bloqueantes;
4. 33_Project_Risk_Register_and_Business_Continuity_Plan.xlsx, dashboard y riesgos críticos;
5. el documento especializado de la tarea.

Incorporación completa

Leer:

1. 00, 01, 02, 03, 04, 05, 06, 07, 10, 11;
2. 14, 15 y este manual;
3. los documentos especializados del rol;
4. 31, 32 y 33;
5. código, configuración, escena, pruebas y registros aplicables.

Transferencia o auditoría

Leer el conjunto 00–34, revisar los artefactos del paquete, verificar hashes, accesos, backups, riesgos, defectos, builds y evidencia, y ejecutar el checklist de transferencia del anexo.

0.5. Estados usados

Estado	Interpretación operativa
HISTORICAL	evidencia o decisión anterior conservada; no describe automáticamente el presente
OBSERVED	presente en la instantánea inspeccionada; debe reverificarse en otra working copy
CURRENT AUTHORITATIVE PLANNED	documento vigente para su ámbito definido, pero no necesariamente implementado ni probado
IMPLEMENTED	existe código/configuración, sin implicar aceptación
VERIFIED	existe evidencia reproducible identificada
PASS	cumple el criterio y no quedan bloqueantes aplicables
FAIL	incumple un criterio
BLOCKED	no puede ejecutarse o aprobarse por una dependencia
ACCEPTED EXCEPTION	desviación aprobada, acotada, con propietario y vencimiento
SUPERSEDED	sustituido por una decisión posterior, conservado por genealogía
UNKNOWN	no debe inferirse; se investiga o se mantiene abierto

0.6. Unidad mínima de handoff

Un handoff válido contiene:

- quién entrega y quién recibe;
- fecha y zona horaria;
- propósito y alcance;
- rama, SHA y estado de working copy;
- versión de Unity, paquetes y versión de aplicación;
- baseline documental y hashes relevantes;
- estado de sprint, gate y próxima acción;
- cambios terminados, en curso y no iniciados;
- tests, build, logs y evidencia;
- defectos, riesgos, excepciones y decisiones pendientes;
- accesos entregados mediante canal seguro;
- instrucciones de rollback y recuperación;
- aceptación explícita del receptor.

Una conversación o nota que omite estos campos es un resumen informal, no una transferencia operativa.

0.7. Estado de esta regeneración dirigida

Esta versión incorpora los resultados reales de la consolidación ejecutada después de RC1. La presencia de un documento no implica que su gate esté cerrado.

Grupo	Estado en esta versión	Próxima dependencia
12–15	regenerados tras RC1	revalidar y refrescar hashes después de los findings finales
21 y 23	regenerados tras RC1	verificar referencias en la auditoría 16 candidata
31	regenerado; 280 controles H6 y 223 controles Release permanecen NOT RUN	ejecutar solo con evidencia real
33	regenerado; 192 riesgos y controles de continuidad siguen abiertos o no verificados	alimentar findings, restauración y handoff
34	regenerado en esta pasada	usar como entrada de 32 candidata y 16 candidata
32	versión RC1 existente, no final	regenerar ahora como manifiesto candidato
16	auditoría RC1 existente	regenerar sobre el conjunto actual 00–34
paquete externo	no generado	crear solo después de resolver findings y congelar contenidos

La secuencia autorizada a partir de esta versión es:

34 actual → 32 candidata → 16 candidata → correcciones → 13/31/33 y fuentes afectadas → 16 final → 32 final → artefactos externos → ZIP → restauración limpia → handoff independiente.

Estado de verdad: H6, Release Candidate, lanzamiento, restauración y handoff independiente permanecen pendientes. Ninguna tabla, plantilla o archivo presente debe transformarse en PASS sin ejecución y evidencia.

1. Identidad, fotografía actual y límites de confianza

1.1. Identidad del producto

Cartridge & Cloud es un simulador de gestión y construcción de una tienda de videojuegos para PC/Steam. Su vertical slice debe demostrar gestión visible, construcción funcional, sistemas comprensibles, clientes observables, progresión acumulativa, claridad de control y una tienda inicial representativa. La visión completa contiene sistemas posteriores —servicios informáticos, empleados, investigación, publishing, desarrollo interno, comercio online y plataforma digital— que no forman parte automática del alcance de H6.

1.2. Fotografía técnica observada

Elemento	Valor observado / documentado
Motor	Unity 6000.3.18f1 (5eb53e4c07)
Render pipeline	URP 17.3.0
Plataforma inicial	Windows x64 / Steam
Compañía	VRM Games
Producto	Cartridge & Cloud
Application Identifier	com.vrmgames.cartridgeandcloud
Versión de aplicación	0.0.21
Input	Input System 1.19.0
Localización	Unity Localization 1.5.12
AI Navigation	2.0.13
Test Framework	1.6.0
Backend de desarrollo observado	Mono / sin override Standalone final aprobado
Build Profile observado	Windows_Development
Resolución por defecto observada	1024 × 768
Ventana redimensionable	desactivada
Player log	activado
GC incremental	activado
Scripts C# del proyecto	457
Escenas propias	5
Prefabs propios	64
Modelos FBX propios	62
Assembly Definitions	12

Los valores anteriores son una fotografía del paquete previo a la regeneración documental. Antes de operar sobre un repositorio activo deben releerse `ProjectVersion.txt`, `manifest.json`, `ProjectSettings`, `Build Profiles` y el SHA real.

1.3. Escenas y roles

Escena	Rol operativo
Bootstrap.unity	arranque, composición inicial y transición segura
MainMenu.unity	menú, slots, opciones y entrada al juego
Store.unity	base funcional histórica y fallback durante la migración
StoreInitial.unity	objetivo representativo de Sprint 16 y futuro slice
TestLab.unity	integración y pruebas controladas

Store no se elimina ni se degrada irreversiblemente hasta que StoreInitial complete conexión runtime, Golden Path, regresión y build externa. StoreInitial no debe reconstruir managers globales ni depender de nombres de GameObject para registrar sistemas.

1.4. Build Profile observado

El perfil Windows_Development contiene históricamente:

1. Bootstrap;
2. MainMenu;
3. Store;
4. TestLab.

StoreInitial forma parte del flujo validado de Sprint 16 y sustituyó a Store como destino representativo. Cualquier cambio posterior de Scene List continúa siendo una operación de gate y debe revalidar rutas de escena, ApplicationRoot, persistencia, UI, input, Golden Path y build externa.

1.5. Estado de sprints y gates

Ámbito	Estado de referencia	Lectura correcta
Sprints 0-15	CLOSED / PASS	capacidades funcionales heredadas; siguen sujetas a regresión
Sprint 16	COMPLETED / PASS	arte/audio representativo, StoreInitial, Golden Path y build externa aceptados

Ámbito	Estado de referencia	Lectura correcta
Sprint 17	PENDING / READY TO OPEN	estabilización; la condición de entrada de S16 está satisfecha, falta kickoff formal
H6	PENDING / BLOCKED	vertical slice no aprobado
Steam onboarding	no demostrado	no asumir App ID, depots, permisos o página operativa
Campaña pública	bloqueada	naming, legal, assets, build, claims y gates siguen abiertos
Release Candidate	no existe	no etiquetar una build local como RC

1.6. Registro operativo de cierre de Sprint 16

El 2026-07-06 se registró el cierre COMPLETED / PASS sobre la build 0.0.21. La evidencia declarada incluye compilación, EditMode, PlayMode, regresión manual, Golden Path externo, persistencia tras reinicio, revisión de Player.log y aprobación visual/funcional. El commit y push de los cambios técnicos se completaron; el SHA exacto debe leerse del repositorio activo y registrarse antes de congelar la baseline de Sprint 17.

1.7. Verdad Git histórica

El handoff v0.6 registró:

- último commit validado: 091090c43855b0b26b09abe9335d18b978ac7eab;
- main observado: d54316c771aab2143993e99b9fd58f2f88016568;
- integración y hotfixes posteriores presentes en working copy;
- obligación de comprobar el SHA real antes de publicar.

Estos identificadores son **históricos**. El ZIP suministrado no contiene .git, por lo que no puede demostrar el HEAD actual, el estado remoto ni la limpieza de la working copy. El receptor debe obtener esa verdad del repositorio conectado.

1.7. Pruebas y límites de la evidencia

La baseline v0.6 documentó 1215 EditMode + 70 PlayMode = 1285 PASS. Esto demuestra el estado de una ejecución concreta, no garantiza:

- que otra working copy compile;
- que StoreInitial esté aprobada visualmente;
- que exista build posterior a la integración;

- que el Golden Path representativo haya pasado;
- que el rendimiento objetivo esté medido;
- que H6 o release estén cerrados.

Cada nueva candidata requiere nueva ejecución, build, smoke externo y evidencia vinculada.

2. Jerarquía documental y resolución de verdad

2.1. Niveles de autoridad

T0 — Constitución del producto

- 00_Enfoque_y_Alcance.md.

T1 — Contratos maestros

- 01–13: diseño, slice, técnica, datos, UX, roadmap, QA, código, setup, builds, producción y cambios.

T2 — Gobierno y navegación

- 14_Project_Binder_Indice_Maestro.md;
- 15_Guia_Maestra.md;
- 16_Auditoria_Global_de_Coherencia.md.

T3 — Autoridades especializadas

- 17–30: arte, audio, UI, economía, contenido, localización, legal, Steam, postlanzamiento, privacidad, seguridad, marketing, accesibilidad y rendimiento.

T4 — Cierre operativo y transferencia

- 31_H6_and_Release_Readiness_Checklist.xlsx;
- 32_Documentation_Baseline_Manifest.xlsx;
- 33_Project_Risk_Register_and_Business_Continuity_Plan.xlsx;
- este manual 34.

2.2. Orden para resolver una pregunta

1. Formular la pregunta y el ámbito.
2. Localizar el documento competente.
3. Comprobar versión, fecha, estado y autoridad.
4. Distinguir objetivo, implementación, prueba, excepción e historia.

5. Contrastar con código/configuración/build cuando la pregunta afecte al estado real.
6. Consultar ADR y registros de sprint cuando se necesite la causa de una decisión.
7. Registrar contradicción o cambio en 13 y el trabajo en 12.
8. Actualizar todos los documentos impactados.
9. ejecutar la verificación correspondiente.
10. cerrar con evidencia, decisión, versión, hash y, cuando aplique, commit/build.

2.3. Reglas de prevalencia

- Alcance constitucional prevalece sobre ideas o backlog no aprobados.
- VSS prevalece sobre descripciones generales al decidir H6.
- Modelo de Datos prevalece sobre ejemplos narrativos en invariantes.
- TDD y código/configuración observados prevalecen sobre una guía histórica para describir la implementación.
- Registro de ejecución vinculado a una build prevalece sobre “funciona en mi máquina”.
- Un gate más estricto prevalece sobre una práctica histórica menos exigente.
- Derecho, seguridad, privacidad y licencias pueden bloquear una decisión comercial o técnica.
- Un riesgo no desaparece porque no haya ocurrido; requiere control, aceptación o cierre verificable.

2.4. Qué hacer ante contradicción

No elegir silenciosamente una de las versiones. Crear una entrada que incluya:

- documentos y líneas/celdas en conflicto;
- código, configuración o evidencia relacionada;
- impacto sobre producto, QA, build, seguridad, legal y comunicación;
- propietario de la decisión;
- decisión temporal si el trabajo no puede detenerse;
- cambio definitivo y documentos a regenerar;
- pruebas de cierre.

2.5. Chats y memoria

Los chats pueden contener contexto útil, pero no son autoridad del proyecto. Una decisión que solo existe en un chat debe:

1. resumirse sin depender del razonamiento privado;
2. registrarse en el documento competente o ADR;
3. propagarse a producción, QA y trazabilidad;
4. validarse;
5. incluirse en el siguiente handoff.

3. Genealogía histórica y decisiones heredadas

3.1. Principio de conservación

Las baselines anteriores son inmutables y conservan por qué se tomaron decisiones, qué se validó y cómo evolucionó el proyecto. No se editan para que coincidan con el presente. Las correcciones se incorporan en una nueva baseline o documento vigente, manteniendo la relación de sustitución.

3.2. Etapas documentales

Etapas	Aporte principal	Uso actual
Material previo a v0.3	primeras definiciones, estructuras y borradores	contexto histórico; no autoridad automática
Baseline v0.3	primer paquete formal completo	genealogía de producto, técnica, QA y negocio
Baseline v0.4	foundation y Sprint 0	evidencia inicial de proyecto Unity y cierre fundacional
Baseline v0.5	Sprints 0–5	primera transferencia extensa, Store funcional y reglas de sprint
Registros vivos Sprints 6–15	sistemas de inventario, proveedores, displays, clientes, checkout, día, economía, save y UI	causa y evidencia de capacidades cerradas
Baseline v0.6	consolidación Sprints 0–16	handoff inmediato previo, estado de StoreInitial y apertura S17
Documentación 00–34	reconstrucción maestra consolidada	baseline normativa de trabajo, pendiente de cierre final

3.3. Handoff v0.5 — valor y sustituciones

El handoff v0.5 registró versión **0.0.6**, commits de Sprint 5, **168/168** tests, Store 10×15 m, grid 20×30, controles de movimiento/construcción y próximo Sprint 6. Sus reglas de trabajo siguen siendo relevantes:

- leer documentos antes de código;
- congelar charter y aceptación;

- preservar `.meta`;
- mantener Domain/Application independientes de Unity;
- exigir compilación/Test Runner del usuario;
- no declarar PASS sin suite y manual;
- cerrar con documentación, QA y trazabilidad;
- no crear tag/release/checksum sin gate.

Han quedado sustituidos como estado actual: versión, commits, conteo de tests y “próximo Sprint 6”. Se conservan como fotografía histórica.

3.4. Handoff v0.6 — valor y límites

El handoff v0.6 registró versión 0.0.17, baseline v0.6, 1285 tests, StoreInitial pendiente y las prohibiciones:

- no cerrar S16 por tests verdes;
- no reimportar FBX sin cambio de fuente;
- no borrar fallbacks todavía;
- no cambiar Build Profiles antes de conectar StoreInitial;
- no depender de nombres de GameObject para registrar sistemas.

Estas reglas continúan vigentes hasta que exista evidencia formal que las sustituya. Los SHA históricos deben verificarse.

3.5. Registros de sprint

Los cierres de Sprints 0–15 no deben reducirse a una tabla de “PASS”. Contienen:

- charters y límites;
- ADR;
- matrices de aceptación;
- checklists y ejecuciones;
- cambios aplicados;
- impacto documental;
- build records;
- deudas y handoffs.

Cuando una regresión afecte a un sistema cerrado, se consulta el sprint que lo introdujo para recuperar su contrato y pruebas originales.

3.6. Fuentes históricas no publicadas como baseline

Los directorios de desarrollo contienen working documents, borradores, arte conceptual, reportes de integración, archivos de producción y fuentes de autoría. Su función es demostrar provenance, proceso o contexto; no sustituyen una baseline publicada. El manifiesto 32 conserva el inventario íntegro de 745 archivos históricos y 643 hashes únicos.

4. Modelo operativo, roles y derechos de decisión

4.1. Operación en estudio unipersonal

Aunque una persona pueda ejercer varios roles, las decisiones deben registrarse como si los roles fueran separables. Esto evita que “lo decidí yo” oculte qué criterio se aplicó. El mismo individuo puede figurar como Product Owner, Engineering y QA, pero debe conservar revisiones distintas y evidencia suficiente.

4.2. Roles mínimos

Rol	Responsabilidad	No puede cerrar por sí solo
Project Owner / Creative Direction	visión, alcance, prioridades, aceptación de producto	QA técnica sin evidencia, licencias sin prueba
Production	secuencia, dependencias, WIP, gates, handoff	cambiar producto sin owner
Engineering	arquitectura, código, integración, diagnósticos	aceptación visual o comercial
Game Design	reglas, balance, contenido y UX	estado de implementación sin prueba
Art / Audio / UI	sistémica assets y presentación	licencias, rendimiento o accesibilidad final sin revisión
QA	estrategia, ejecución, defectos y recomendación de gate	aceptar riesgo de negocio no delegado
Build / Release	candidatos, manifests, hashes y distribución	redefinir criterios de aceptación
Documentation	autoridad, coherencia, baseline y cambios	inventar estado técnico
Security / Privacy	accesos, secretos, incidentes y datos	aceptar exposición crítica sin owner
Legal / Publishing	derechos, notices, Steam y claims	aprobar contenido sin evidencia de titularidad

4.3. Derechos de decisión

- Product Owner aprueba alcance, identidad, prioridades y excepciones de producto.
- Engineering aprueba solución técnica dentro de contratos; cambios arquitectónicos transversales requieren ADR.

- QA puede bloquear un gate por evidencia insuficiente o defecto bloqueante.
- Security/Legal puede detener distribución o acceso ante un riesgo crítico.
- Production coordina, pero no convierte una recomendación en PASS sin signoff competente.
- Release custodia artefactos; no modifica una candidata después del hash.

4.4. Sustitución y escalado

Cada función crítica debe tener:

- titular;
- sustituto o procedimiento de emergencia;
- accesos mínimos;
- documentación de operación;
- criterio de escalado;
- canal seguro de contacto.

El libro 33 contiene roles de crisis y dependencias. Este manual define cómo transferir conocimiento; no almacena contraseñas, recovery codes ni secretos.

4.5. Principio de mínimo privilegio

Un colaborador recibe solo:

- repositorios necesarios;
- ramas y permisos acordes al alcance;
- carpetas o canales relevantes;
- cuentas temporales cuando sea posible;
- acceso a builds no públicas según necesidad;
- datos anonimizados o sintéticos.

Al terminar se revocan accesos, se rotan secretos compartidos, se recuperan activos y se registra el offboarding.

5. Onboarding y retoma por niveles

5.1. Orientación de 15 minutos

Objetivo: impedir una acción peligrosa antes de entender el estado.

1. Leer la portada y secciones 1–3.
2. Abrir dashboard de 31, 32 y 33.
3. Confirmar que Sprint 16 está cerrado y que Sprint 17/H6 mantienen el estado registrado, salvo evidencia posterior.
4. Identificar la rama, SHA y working copy reales.

5. No editar escenas, Build Profiles, paquetes, versiones o documentos de gobierno todavía.
6. Elegir el documento especializado del rol.

5.2. Incorporación de dos horas

1. Leer 00, 02, 03, 06, 07, 10, 11, 14, 15, 34.
2. Leer el documento especializado.
3. Recorrer estructura de Assets y Documentation sin modificar.
4. Abrir Unity con la versión exacta.
5. Comprobar compilación, Console, escenas y paquetes.
6. Revisar tests y defectos abiertos.
7. Ejecutar un recorrido seguro o TestLab, no una modificación amplia.
8. Registrar preguntas y contradicciones.

5.3. Incorporación de un día

Debe producir:

- entorno validado;
- repositorio y permisos comprobados;
- lectura del contrato de tarea;
- ejecución de tests base relevante;
- una build o escenario de referencia cuando corresponda;
- entendimiento de rollback;
- tarea inicial pequeña y reversible;
- primer handoff de aprendizaje.

5.4. Ruta de ingeniería

Lectura mínima adicional: 03, 04, 09, 10, 11, 27, 29, 30; assemblies y tests del sistema. Antes de tocar código, localizar invariantes, IDs, resultados tipados, mutaciones atómicas, persistencia y cobertura.

5.5. Ruta de arte/audio/UI

Lectura mínima: 02, 05, 10, 17, 18, 19, 21, 22, 23, 29, 30. Validar naming, pivots, escalas, import settings, licencias, LOD, materiales, variantes, texto, foco, audio y budgets.

5.6. Ruta de QA/release

Lectura mínima: 02, 07, 08, 11, 24, 27, 29, 30, 31, 33. Debe comprender severidades, niveles de prueba, evidencia, build identity, logs, Golden Path, integridad y rollback.

5.7. Ruta de producción/documentación

Lectura mínima: 00, 06, 12–16, 31–34. Debe mantener una única lista de trabajo, cambios, riesgos, decisiones, estado de gate y baseline.

5.8. Primera tarea recomendada

La primera contribución de un nuevo colaborador debe ser:

- acotada;
 - reversible;
 - sin migración de schema;
 - sin cambio de paquetes o Build Profile;
 - cubierta por prueba o checklist;
 - revisable en menos de una sesión;
 - acompañada de un mini-handoff.
-

6. Preparación reproducible del entorno

6.1. Requisitos

- Windows compatible con Unity 6.3 LTS y build Windows x64.
- Unity Hub.
- Unity 6000.3.18f1 con módulo Windows Build Support.
- Git o GitHub Desktop según flujo autorizado.
- IDE compatible; Visual Studio Integration 2.0.26 está en el proyecto.
- espacio suficiente para Library, builds, backups y caches.
- acceso al repositorio y, cuando proceda, Git LFS.

6.2. Obtención del repositorio

1. Clonar mediante canal aprobado; no trabajar directamente dentro de un ZIP.
2. Elegir una ruta local corta, estable y sin sincronización cloud agresiva.
3. Confirmar rama y remoto.
4. Ejecutar `status` o comprobar GitHub Desktop antes de abrir Unity.
5. Verificar que `.meta` están presentes.
6. Verificar LFS antes de abrir assets binarios si el repositorio lo usa.
7. Registrar SHA inicial y estado de working copy.

6.3. Configuración crítica de Unity

Deben mantenerse:

- Version Control: Visible Meta Files;
- Asset Serialization: Force Text;

- finales de línea y `.gitattributes` del repositorio;
- paquetes según `manifest.json` y `lockfile`;
- Input System activo;
- URP configurado;
- no reserializar el proyecto completo sin aprobación.

6.4. Primera apertura

1. Abrir con la versión exacta.
2. Esperar importación completa.
3. Si aparece Safe Mode, no pulsar ignorar sin capturar errores.
4. Guardar Console y paquetes problemáticos.
5. No hacer Reimport All como primera respuesta.
6. Resolver en orden: versión de Unity, paquetes, scripts, `asmdefs`, `assets/GUID`, escenas.
7. Confirmar ausencia de cambios masivos inesperados en Git.

6.5. Validación inicial

Comprobar:

- `ProjectVersion.txt`;
- paquetes y versiones;
- compilación sin errores;
- cinco escenas propias;
- Build Profile esperado;
- `ApplicationRoot` y composición;
- catálogos y Resources necesarios;
- tests `EditMode/PlayMode` relevantes;
- escena de trabajo;
- ruta de save de prueba;
- `Player.log` en build cuando corresponda.

6.6. Prohibiciones de setup

- no actualizar Unity o paquetes por comodidad;
- no borrar `Library` como sustituto de diagnóstico;
- no aceptar reserialización masiva sin revisar diff;
- no generar nuevos GUID para arreglar referencias;
- no copiar assets sin `.meta`;
- no activar servicios online o analytics sin revisión de privacidad;
- no instalar plugins de origen no verificado;
- no guardar secretos en Assets, `ProjectSettings`, scripts o documentación versionada.

7. Repositorio, ramas, commits y working copy

7.1. Fuente de verdad de código

La fuente de verdad es el repositorio remoto aprobado más el SHA identificado de la working copy. Un ZIP sirve como snapshot o transferencia, pero no demuestra historia, ramas, tags ni remoto.

7.2. Rama estable

`main` es la rama estable esperada. No implica que todo commit en `main` sea un release. Debe mantenerse integrable y asociable a evidencia.

7.3. Ramas de trabajo

Crear una rama cuando:

- el cambio dure más de una sesión;
- afecte arquitectura, schema, escenas o assets compartidos;
- requiera revisión o experimento;
- pueda bloquear `main`.

Una corrección minúscula puede hacerse en el flujo aprobado, pero nunca se publica sin petición y revisión explícitas.

7.4. Working copy limpia

Antes de comenzar:

- registrar cambios existentes;
- identificar archivos no trackeados;
- separar trabajo ajeno;
- no usar `discard` o reset destructivo sin backup;
- no mezclar regeneración documental con cambios de juego si puede evitarse.

Al finalizar, la working copy debe quedar limpia o acompañada de un handoff que describa cada cambio.

7.5. Commits

Un commit debe ser:

- intencional;
- coherente;
- compilable cuando aplica;
- pequeño para revisar;
- descrito por resultado, no por actividad;
- acompañado de tests/evidencia adecuados.

Formato recomendado:

<tipo>(<ámbito>): <resultado observable>

Por qué:

- ...

Validación:

- ...

Impacto documental/riesgo:

- ...

7.6. Push, merge, tag, identidad Git y release

- no hacer push, merge, tag o release sin autorización explícita del propietario;
- verificar remoto, rama, upstream y working copy antes de publicar;
- no force-push a una rama compartida salvo incidente controlado;
- los tags se reservan para gates definidos y deben apuntar a un commit verificable;
- una release requiere build inmutable, manifiesto, hashes, evidencias, notas y rollback.

Durante el cierre documental se distinguen tres identidades:

Identidad	Qué representa	Dónde debe registrarse
PROJECT_SOURCE_COMMIT	commit exacto del proyecto Unity/código auditado	13, 32, 34 y PACKAGE_MANIFEST.md
DOCUMENTATION_TAG	nombre del tag que identifica la baseline documental	13, 32, 34 y manifest externo
DOCUMENTATION_COMMIT_SHA	commit que contiene la baseline documental final	manifest externo, annotated tag, release metadata o registro posterior

No se debe intentar introducir dentro de un archivo el SHA-256 del propio archivo ni el SHA del mismo commit que lo contiene: ambos casos son autorreferenciales. El commit del proyecto auditado sí puede y debe registrarse internamente. El commit documental definitivo se conserva externamente o en el propio objeto tag.

Campos pendientes no deben inventarse. Hasta que existan, usar expresamente:

- `PROJECT_SOURCE_COMMIT`: PENDING VERIFICATION;
 - `DOCUMENTATION_TAG`: PENDING APPROVAL;
 - `DOCUMENTATION_COMMIT_SHA`: EXTERNAL AFTER COMMIT. ## 7.7. Archivos Unity
 - preservar `.meta` junto al asset;
 - evitar edición simultánea de escenas/prefabs grandes;
 - revisar diffs YAML con cautela;
 - no resolver conflictos de escena eligiendo “ours/theirs” sin reconstruir intención;
 - reabrir y validar tras un merge de escenas, prefabs, ProjectSettings o paquetes.
-

8. Estado operativo y próxima secuencia autorizada

8.1. Estado heredado de Sprint 16

Cerrado dentro de Sprint 16:

- fase funcional;
- placeholders representativos;
- audio/VFX iniciales;
- catálogos;
- integration tests;
- Golden Path y builds previos;
- importación representativa, prefabs, LOD transfer y runtime catalog link.

No aceptado en el handoff v0.6:

- composición de muros, puerta y mobiliario de warehouse;
- presentación visual final de `StoreInitial`;
- build posterior a la integración;
- cierre formal del sprint.

8.2. Plan heredado de `StoreInitial`

La secuencia histórica de 11 pasos es:

1. crear `StoreInitial.unity` como copia de Store funcional;
2. crear `StoreInitialEnvironment.prefab`;
3. montar suelo y arquitectura manualmente;
4. configurar puerta manualmente;

5. colocar mobiliario inicial;
6. añadir anchors y roots técnicos;
7. crear `StoreInitialSceneContext`;
8. conectar runtime;
9. desactivar generación procedural del shell;
10. corregir supresión de clic sobre UI;
11. validar tests, Golden Path y build.

Jerarquía objetivo:

```
StoreInitialEnvironment
├─ Architecture
├─ InitialFurniture
├─ Lighting
├─ Anchors
└─ TechnicalColliders
```

No incluir dentro del prefab de entorno: `EventSystem`, HUD, cámara, managers globales, save services o `ApplicationRoot`.

8.3. Regla de continuación

Antes de asumir qué paso está ejecutado:

1. revisar Git/working copy;
2. abrir `StoreInitial`;
3. inspeccionar jerarquía y referencias;
4. ejecutar validaciones existentes;
5. comparar con registros de Sprint 16;
6. actualizar `Current Status`.

No repetir trabajo basándose solo en el plan ni declarar terminado lo que solo aparece como previsto.

8.4. Cierre de Sprint 16

Requiere, como mínimo:

- `StoreInitial` autorada y aprobada visualmente;
- runtime conectado sin duplicidad;
- procedural shell desactivado donde corresponda;
- UI/input corregidos;
- tests completos aplicables;
- Golden Path manual;
- build externa posterior a integración;
- revisión de logs;
- documentación, trazabilidad, defectos y handoff actualizados.

8.5. Sprint 17

Se abre después del cierre S16. Su ámbito es balance, rendimiento, QA, bug fixing, revisión de accesibilidad, build y cierre del vertical slice. No debe introducir sistemas mayores nuevos.

8.6. H6

H6 se decide mediante 02 y 31. Tests verdes no bastan. Deben cerrarse criterios, evidencias, defectos, rendimiento, accesibilidad, legal, build, seguridad, documentación y signoffs aplicables.

9. Ciclo operativo de una tarea o cambio

9.1. Entrada

Toda tarea debe tener:

- ID;
- resultado esperado;
- fuente/criterio;
- alcance y fuera de alcance;
- owner y reviewer;
- dependencias;
- riesgo;
- prueba/evidencia;
- impacto documental;
- rollback.

9.2. Preparación

1. leer fuentes;
2. comprobar working copy;
3. congelar criterio de aceptación;
4. reproducir estado base;
5. decidir rama;
6. identificar tests existentes;
7. capturar evidencia inicial cuando sea una corrección;
8. registrar riesgo o ADR si procede.

9.3. Implementación

- cambio mínimo coherente;
- respetar capas e invariantes;
- validar antes de mutar;

- mantener atomicidad e idempotencia;
- evitar búsquedas frágiles de escena;
- preservar IDs y compatibilidad;
- no mezclar refactors no necesarios;
- mantener accesibilidad, localización, rendimiento y seguridad.

9.4. Verificación

- prueba unitaria/determinista;
- integración;
- escena objetivo;
- manual/Golden Path;
- build externa cuando aplica;
- logs y comparación;
- regresión de sistemas vecinos;
- evidencia archivada.

9.5. Cierre

- criterio PASS o excepción aprobada;
- defectos registrados;
- producción y trazabilidad actualizadas;
- documentación viva actualizada;
- commit identificado;
- handoff actualizado;
- rollback conocido;
- cambios no relacionados excluidos.

9.6. Definition of Done transversal

Una tarea no está terminada si falta cualquiera de los elementos aplicables:

- implementación integrada;
 - aceptación funcional;
 - test y evidencia;
 - experiencia/arte revisados;
 - rendimiento sin regresión;
 - accesibilidad/localización revisadas;
 - licencias/provenance;
 - seguridad/privacidad;
 - documentación/trazabilidad;
 - build o paquete cuando corresponde.
-

10. Arquitectura y organización del proyecto

10.1. Raíz propia

El contenido del juego reside bajo `Assets/_Project`. Los assets de terceros deben permanecer identificables y no mezclarse sin provenance.

10.2. Organización funcional

- Animations;
- Art (Materials, Models, Sprites, Textures, VFX);
- Audio (Ambience, Music, SFX, UI);
- Content (Localization, Placement, ScriptableObjects);
- Data (Catalogs, Checkout, Customers, DayCycle, Displays, Economy, Persistence, Placement, Products, Shopping, Store, Suppliers);
- Editor;
- Prefabs;
- Reports;
- Resources;
- Scenes;
- Scripts (Application, Domain, Infrastructure, Presentation, Runtime);
- Settings;
- Tests;
- UI.

10.3. Capas

- Domain: reglas puras, entidades, value objects e invariantes.
- Application: casos de uso y coordinación sin dependencia de presentación.
- Infrastructure: persistencia, adaptadores y servicios externos.
- Presentation: UI y control de interacción.
- Runtime: composición Unity y lifecycle.
- Editor: tooling y validación de authoring.
- Tests: cobertura por nivel.

Las dependencias apuntan hacia el núcleo. No introducir Unity en Domain/Application para resolver una conveniencia local.

10.4. Composición

`ApplicationRoot` y composition roots crean y conectan servicios. Las escenas son composición y autoría, no bases de datos. Los registros no deben depender de nombres de `GameObject`.

10.5. Datos e IDs

- IDs estables y únicos;

- ScriptableObjects para datos de authoring;
- estado runtime separado;
- catálogos validados;
- cambios de ID tratados como migración peligrosa;
- contenido representativo vinculado a provenance y catálogo 21.

10.6. Persistencia

Antes de cambiar schema:

- definir compatibilidad;
 - preparar migración;
 - conservar saves de prueba;
 - probar carga/guardado/recuperación;
 - actualizar versión y documentos;
 - impedir invalidación silenciosa.
-

11. QA, defectos, evidencia y gates

11.1. Niveles de prueba

- A: determinista/automática;
- B: integración en TestLab;
- C: integración en StoreInitial;
- D: Golden Path manual;
- E: build externa;
- F: sesión extensa.

La capa adecuada depende del riesgo. Una prueba de dominio no sustituye una comprobación de escena, y una captura de escena no sustituye invariantes.

11.2. Ejecución

Cada ejecución registra:

- build/commit;
- entorno;
- caso y datos;
- resultado;
- evidencia;
- defectos;
- ejecutor y fecha;
- repetibilidad.

11.3. Defectos y deuda

No mezclar conteos sin explicar el universo. QA puede contar defectos; producción puede incluir deuda o configuración bloqueante. Cada elemento debe tener severidad, prioridad, owner, estado, reproducción, impacto y gate.

11.4. Evidencia

Evidencia aceptable:

- XML/resultado de tests;
- logs identificados;
- capturas o vídeo con build ID;
- save before/after;
- profiler capture;
- manifest/hash;
- checklist firmado;
- registro de ejecución.

La evidencia se almacena fuera de carpetas temporales y se referencia por ID estable.

11.5. Checklist 31

El libro 31 contiene 266 controles H6, 209 controles de release, 71 pruebas y 40 paquetes de evidencia. Sus estados iniciales NOT RUN/MISSING/OPEN son deliberados. No convertir filas en PASS sin evidencia.

11.6. Excepciones

Una excepción requiere:

- criterio incumplido;
 - razón;
 - riesgo residual;
 - scope afectado;
 - mitigación;
 - propietario;
 - fecha de expiración/revisión;
 - aprobación competente;
 - prohibición de ocultarla en release notes o claims.
-

12. Builds, versionado y distribución

12.1. Ejes de versión

Mantener separados:

- versión de aplicación;
- schema de persistencia;
- versión de contenido;
- baseline documental;
- Unity/paquetes;
- build ID;
- commit SHA;
- canal/branch.

No colapsar todos los cambios en un único número.

12.2. Tipos de build

- Development: diagnóstico local.
- QA: ejecución controlada.
- Milestone candidate: candidata a sprint/hito.
- Release Candidate: artefacto congelado para decisión de release.
- Release: distribución aprobada.

El nombre de la carpeta no cambia el tipo real.

12.3. Preflight

Comprobar:

- rama/SHA y working copy;
- versión;
- Unity y paquetes;
- Build Profile y escenas;
- scripting backend/arquitectura;
- datos y catálogos;
- tests requeridos;
- defectos y excepciones;
- legal/licencias/notices;
- seguridad/secretos;
- espacio y ruta de salida.

12.4. Build externa

Después de Build Succeeded:

1. ejecutar fuera del Editor;
2. validar arranque y menú;

3. crear/cargar slot;
4. ejecutar recorrido requerido;
5. cerrar y reabrir;
6. revisar Player.log;
7. conservar build y evidencia;
8. calcular hashes en candidatos.

12.5. Inmutabilidad

Después de calcular hashes de una candidata, no modificar archivos. Un cambio exige nueva build ID y nuevos hashes.

12.6. Steam

No subir ni publicar hasta completar onboarding, permisos, depots, ramas, store assets, legal, privacidad, seguridad, QA, accesibilidad y rollback. Seguir 24, 28 y 31.

13. Operación documental y baseline

13.1. Documentación viva

Actualizar el documento competente en el mismo cambio cuando se modifica una regla, arquitectura, flujo, build, configuración, gate o procedimiento. No esperar al final del proyecto para reconstruir decisiones.

13.2. Control de cambios

13_Trazabilidad_y_Control_de_Cambios.xlsx debe registrar:

- Change ID;
- causa y decisión;
- archivos/documentos afectados;
- tests;
- riesgos;
- versión/commit;
- aprobación y cierre.

Toda regeneración de 16, 32, 33, 34 o un artefacto externo invalida los hashes y referencias dependientes hasta que se recalculen.

13.3. Baselines

Una baseline es un paquete identificado e inmutable. Debe contener:

- documentos aprobados;

- manifest;
- checksums;
- informe de validación;
- README;
- relaciones/sustituciones;
- fecha y versión;
- responsable;
- ubicación de archivo y restauración.

La carpeta de trabajo RC1 y la carpeta de documentos regenerados son fuentes de consolidación, no una baseline final.

13.4. Manifiesto 32

32_Documentation_Baseline_Manifest.xlsx tiene dos estados distintos:

1. **candidato:** inventaría el conjunto actual, marca pendientes y permite ejecutar la auditoría 16 candidata;
2. **definitivo:** se genera después de resolver findings y cuando no quedan cambios de contenido previstos.

La versión RC1 de 32 no es final. La próxima acción es regenerarla como candidata incorporando 12–15, 21, 23, 31, 33 y esta versión de 34.

13.5. Autorreferencia de hashes y Git

Un archivo no puede estabilizar su propio SHA-256 dentro de sí mismo. Por tanto:

- el SHA-256 de 32, 33 y 34 se calcula después de exportarlos;
- dichos hashes se registran en CHECKSUMS_SHA256.txt, PACKAGE_MANIFEST.md y los registros que puedan actualizarse sin crear una nueva autorreferencia;
- el checksum del ZIP se publica fuera del propio ZIP;
- PROJECT_SOURCE_COMMIT puede registrarse dentro de los documentos;
- DOCUMENTATION_COMMIT_SHA se registra externamente o mediante un tag anotado.

13.6. Estado y secuencia de cierre

Completado antes de esta versión:

1. copia candidata RC1 conservada;
2. regeneración de 12, 13, 14, 15;
3. actualización de 21 y 23;
4. regeneración de 31;
5. regeneración de 33;
6. actualización dirigida de 34 — este documento.

Pendiente en orden:

1. generar 32_Documentation_Baseline_Manifest.xlsx como **candidato**;
2. regenerar 16_Auditoria_Global_de_Coherencia.md como **auditoría candidato**;
3. registrar findings en 13, controles bloqueantes en 31 y riesgos en 33;
4. corregir los documentos fuente y dependientes;
5. regenerar 16 como auditoría final;
6. regenerar 32 como manifiesto definitivo;
7. ejecutar una pasada final limitada de 13, 31, 33 y 34 si los findings o datos reales lo exigen;
8. generar PACKAGE_MANIFEST.md, CHECKSUMS_SHA256.txt, DOCUMENTATION_VALIDATION_REPORT.md, VERSION.txt, README_FIRST.md, BASELINE_CHANGELOG.md y RESTORE_AND_VERIFY.md;
9. crear el ZIP inmutable desde una lista blanca;
10. verificar extracción, inventario, hashes, apertura y fórmulas;
11. asociar el paquete al commit del proyecto y al tag documental;
12. ejecutar restauración limpia y handoff independiente.

No cerrar G-BASE ni G-DCC-RESTORE mientras exista un finding BLOCKER o HIGH sin resolver o sin excepción formal aprobada. *** # 14. Arte, audio, UI, contenido y provenance

14.1. Ingesta de assets

Antes de importar:

- identificar fuente, autor, licencia y restricciones;
- guardar original fuera del asset procesado cuando proceda;
- definir naming, escala, pivote, orientación y unidades;
- decidir compresión, materiales, LOD, colliders y variantes;
- verificar que no contiene datos o marcas no autorizadas.

14.2. Reimportación

No reimportar FBX representativos sin cambio de fuente o motivo documentado. Una reimportación puede alterar materiales, animaciones, rigs, escalas, GUIDs o overrides.

14.3. Prefabs y escenas

- prefab source claro;
- overrides revisados;
- no duplicar managers;
- colliders y navegación coherentes;
- IDs/catálogos vinculados;
- variantes controladas;
- prueba en escena y build.

14.4. Audio

Conservar archivos fuente, licencias, import settings, canales, loudness y mixer plan. No asumir que un clip integrado está licenciado para distribución.

14.5. UI y localización

Toda UI debe probarse con:

- escalas 80–150 %;
- ES/EN;
- texto largo;
- teclado/mando donde corresponda;
- foco y cancelación;
- confirmaciones;
- contraste y alternativas al color;
- resoluciones objetivo.

14.6. Catálogos y registro legal

21 identifica contenido; 23 identifica derechos. Un asset no está listo si solo existe en Assets pero no tiene provenance, owner o estado de licencia.

15. Accesibilidad, localización y rendimiento como operaciones continuas

15.1. Accesibilidad

No se trata como revisión final. Cada cambio de UI, input, cámara, VFX, audio, tutorial, mensaje o timing debe evaluar barreras visuales, auditivas, motoras y cognitivas.

Capacidades observadas incluyen escalado UI/texto 80–150 %, reducción de movimiento, tutorial y confirmaciones persistentes. No declarar como completas remapeo, mando integral, subtítulos, narración, contraste configurable o claims Steam sin implementación y prueba.

15.2. Localización

- mantener strings fuera de código cuando corresponda;
- conservar claves estables;
- evitar concatenación no localizable;
- validar fuentes y glifos;
- probar truncado y layout;
- revisar terminología;

- no publicar idiomas sin cobertura y QA.

15.3. Rendimiento

Objetivos históricos: 60 FPS a 1080p, hasta 8 clientes, carga <5 s, save/cierre <2 s y allocations próximas a cero en steady state. Son targets pendientes de benchmark representativo, no claims.

Toda optimización sigue: reproducción → baseline → hipótesis → cambio mínimo → comparación → regresión funcional/visual/accesible.

15.4. Hardware y requisitos de Steam

No publicar mínimos/recomendados hasta medir Player externo en escenarios congelados y hardware definido. El equipo de desarrollo no equivale a hardware mínimo.

16. Seguridad, privacidad, cuentas y secretos

16.1. Secretos

Nunca almacenar en repositorio o documentación:

- contraseñas;
- tokens;
- API keys;
- recovery codes;
- cookies de sesión;
- claves de firma;
- credenciales Steam;
- datos personales no necesarios.

Usar gestor de contraseñas y canales seguros. Versionar únicamente referencias o nombres de secreto.

16.2. Cuentas críticas

Para GitHub, Steam, correo, dominio, almacenamiento, herramientas y proveedores:

- MFA;
- métodos de recuperación;
- propietario y sustituto;
- permisos mínimos;
- inventario de sesiones/aplicaciones;
- revisión periódica;

- offboarding inmediato.

16.3. Dependencias

Actualizar dependencias mediante cambio controlado, no automáticamente. Revisar release notes, compatibilidad, vulnerabilidades, licencias, lockfile, tests y rollback.

16.4. Logs y datos

No compartir logs públicamente sin revisar rutas, nombres de usuario, IDs, hardware, direcciones, tokens o datos de save. La telemetría propia no está abierta por defecto.

16.5. Incidente

Ante sospecha de compromiso:

1. contener;
2. preservar evidencia;
3. revocar/rotar;
4. evaluar alcance;
5. recuperar desde fuente limpia;
6. documentar;
7. comunicar según legal/privacidad;
8. revisar controles.

Seguir 27 y playbooks de 33.

17. Backups, continuidad y recuperación

17.1. Principio

Git remoto no es el único backup. La continuidad exige copias independientes de código, assets, documentación, builds, licencias, saves de prueba, manifests y evidencias.

17.2. Política mínima

Aplicar 3-2-1 cuando sea viable:

- al menos tres copias;
- dos medios o dominios de fallo;
- una copia fuera del equipo principal;
- una copia protegida contra borrado/ransomware cuando sea posible.

17.3. Activos críticos

- repositorio con historia;
- assets fuente y `.meta`;
- documentación 00-34 e histórico;
- ProjectSettings/Packages;
- catálogos y datos;
- licencias/recibos/notices;
- builds candidatas y símbolos controlados;
- saves y corpus de prueba;
- credenciales y recovery codes en gestor seguro;
- arte comercial y store assets.

17.4. RTO/RPO

El libro 33 define BIA y objetivos iniciales. Son targets pendientes de ejercicio. No declarar un RTO/RPO cumplido hasta restaurar en entorno limpio y medir.

17.5. Prueba de restauración limpia

Una copia se considera verificada únicamente cuando se ejecuta **RB-16** y queda evidencia de:

1. selección de un destino limpio sin archivos previos;
2. obtención del ZIP, manifest y checksum externo;
3. verificación del checksum del ZIP antes de extraer;
4. extracción sin sobrescritura ni reparación silenciosa;
5. comparación de rutas, nombres, cantidad y tamaños;
6. verificación SHA-256 de todos los artefactos;
7. apertura de cada XLSX y comprobación de hojas/fórmulas clave;
8. apertura de Markdown/texto y validación de UTF-8, tablas y bloques de código;
9. registro de operador, tiempo, RPO real, defectos y evidencias;
10. signoff o cuarentena del paquete.

Una copia existente, una extracción parcial o la apertura de un único archivo no constituyen **PASS**.

17.6. Pérdida total del equipo o del contexto operativo

Prioridad:

1. proteger a las personas y asegurar cuentas/credenciales;
2. activar al coordinador de recuperación y al sustituto designado;
3. obtener hardware y red alternativos;
4. restaurar gestor de contraseñas/MFA por el mecanismo aprobado;
5. recuperar repositorio, refs, commit/tag del proyecto y paquetes;
6. recuperar el último paquete documental verificado;

7. instalar la versión exacta de Unity;
8. ejecutar RB-16 para documentación y el runbook técnico aplicable;
9. validar integridad, tests, build y estado de riesgos;
10. ejecutar RB-17 para demostrar que otra persona puede retomar;
11. reanudar solo después de un status report y una decisión de autoridad.

Los objetivos RTO/RPO de 33 son targets pendientes de prueba, no promesas demostradas. *** # 18. Gestión de riesgos, incidentes y crisis

18.1. Registro 33

El libro 33 actualizado contiene 192 riesgos y 192 acciones, 20 capacidades BIA, 94 estrategias de continuidad, 19 activos de backup, 30 dependencias críticas, 24 playbooks, 170 pasos de runbook y 72 instancias de ejercicios. El estado abierto, planificado o no verificado no significa que todos los incidentes hayan ocurrido; significa que los controles no deben asumirse eficaces sin evidencia.

18.2. Revisión de riesgo

- semanal para riesgos de sprint/gate;
- mensual para cartera general;
- antes de hito/release;
- después de incidentes o cambios mayores;
- cuando una dependencia, cuenta, licencia o proveedor cambia.

18.3. Disparadores

Un trigger debe ser observable: tests fallan, acceso perdido, backup no restaura, licencia ausente, build corrupta, dependencia vulnerable, desviación de schedule, frame time excedido, save corrupto, secreto filtrado.

18.4. Aceptación

No usar “aceptado” como sinónimo de ignorado. Requiere autoridad, exposición residual, justificación, mitigación, alcance, fecha de expiración y revisión.

18.5. Comunicación de crisis

- una persona coordina;
- hechos y tiempos, no especulación;
- canal separado para decisiones;
- no publicar información sensible;
- conservar timeline;
- actualizar a stakeholders según impacto;
- postmortem sin culpabilización y con acciones.

19. Marketing, publishing, comunidad y postlanzamiento

19.1. Estado

Marketing y comunicación permanecen en planificación interna. No se debe abrir campaña pública basándose únicamente en documentación o concept art.

19.2. Claims

Todo claim debe ser:

- verdadero en build identificada;
- respaldado por evidencia;
- legalmente seguro;
- localizado;
- accesible;
- coherente con scope.

No afirmar “optimizado”, “totalmente accesible”, “compatible con mando”, “Steam Deck”, idiomas o fecha sin gate.

19.3. Assets comerciales

Capturas, GIF, tráiler, key art y cápsulas se producen desde una build/escena aprobada, sin debug, placeholders, marcas no autorizadas ni UI falsa.

19.4. Comunidad y soporte

Antes de abrir Discord, newsletter, foros o soporte:

- owner y moderación;
- privacidad y retención;
- seguridad de cuentas;
- SLA realista;
- políticas y escalado;
- calendario sostenible;
- procedimiento de incidentes.

19.5. Postlanzamiento

El plan 25 aplica cuando exista producto lanzado. Antes, se usa para preparar clasificación de incidencias, hotfix, rollback, soporte, notas y compatibilidad de saves.

20. Ritmos de operación

20.1. Inicio de sesión de trabajo

- comprobar calendario/prioridad;
- Git status/branch/SHA;
- leer handoff;
- revisar bloqueantes y riesgos;
- abrir documento/criterio;
- definir resultado de la sesión;
- asegurar backup de cambios no versionados.

20.2. Cierre de sesión

- guardar y revisar Console;
- ejecutar validación proporcional;
- revisar diff;
- actualizar tarea/defecto/riesgo;
- registrar cambios no commitados;
- crear handoff breve;
- asegurar copia/push solo si autorizado.

20.3. Revisión semanal

- progreso vs gate;
- WIP y dependencias;
- defectos S0/S1;
- riesgos fuera de tolerancia;
- estado de backups y accesos;
- documentación desactualizada;
- build/test de referencia;
- próximo objetivo verificable.

20.4. Cierre de sprint

- scope completo o excepciones;
- suite y manual;
- build cuando corresponde;
- defectos/deuda;
- ADR;
- documentación/trazabilidad;
- versionado;
- handoff;
- signoff.

20.5. Revisión mensual

- restauración o ejercicio programado;
 - cuentas/MFA;
 - dependencias/licencias;
 - almacenamiento y costes;
 - roadmap y riesgos;
 - baseline/documentación;
 - salud del repositorio.
-

21. Procedimientos de diagnóstico y playbooks

21.1. Unity no abre o entra en Safe Mode

1. capturar errores;
2. confirmar versión exacta;
3. comprobar manifest/lockfile;
4. revisar cambios recientes;
5. resolver primer error de compilación, no la cascada;
6. evitar Reimport All;
7. restaurar desde commit/copia si hay corrupción;
8. documentar causa y corrección.

21.2. Cambios masivos inesperados

- cerrar Unity;
- no commit;
- identificar causa: versión, Force Text, line endings, paquete, reimport;
- guardar diff/listado;
- restaurar o aislar;
- reabrir solo con configuración correcta.

21.3. Referencias rotas o **.meta** ausente

- detener edición;
- recuperar **.meta** original desde Git/backup;
- no generar uno nuevo salvo reemplazo intencional;
- validar GUID, prefabs, escenas y catálogos;
- ejecutar regresión.

21.4. Tests pasan pero experiencia falla

- no cerrar;
- reproducir en escena/build;
- crear defecto de integración/aceptación;

- añadir test adecuado si es automatizable;
- revisar si el criterio requería nivel C–F.

21.5. Build falla

- conservar Editor.log/Build report;
- identificar fase y primer error;
- confirmar perfil, escenas, espacio, ruta y permisos;
- revisar cambios en paquetes/ProjectSettings;
- reproducir con build mínima si es necesario;
- no borrar evidencia.

21.6. Save corrupto

- detener escrituras sobre el original;
- copiar archivo y logs;
- identificar versión/schema/build;
- probar carga en entorno aislado;
- aplicar recuperación documentada;
- registrar pérdida y causa;
- añadir corpus/regresión.

21.7. Regresión de rendimiento

- congelar escenario y hardware;
- comparar build anterior;
- capturar profiler/memory/frame debugger;
- localizar CPU/GPU/I/O/GC;
- revertir o corregir cambio mínimo;
- verificar calidad y accesibilidad.

21.8. Credencial comprometida

- revocar sesión/token;
- rotar secreto;
- revisar logs y alcance;
- proteger recovery channels;
- buscar exposición en repositorio/build/logs;
- notificar según plan;
- recuperar desde entorno limpio.

21.9. Pérdida de repositorio/remoto

- congelar working copies;
- identificar copia con historia más completa;
- verificar integridad;

- crear remoto nuevo controlado;
- restaurar ramas/tags;
- comparar assets/LFS;
- rotar credenciales;
- documentar incidente.

21.10. Problema legal o licencia ausente

- detener distribución del asset/build;
 - localizar provenance y alcance;
 - sustituir, retirar o regularizar;
 - actualizar registro 23, créditos y builds;
 - limpiar materiales públicos si procede;
 - conservar evidencia de decisión.
-

22. Protocolo de handoff

22.1. Cuándo es obligatorio

- cambio de chat o herramienta con pérdida de contexto;
- pausa superior a una semana;
- cambio de persona o rol;
- fin de sprint/fase;
- antes de vacaciones o indisponibilidad;
- antes/después de incidente;
- entrega a publisher, contractor o soporte;
- transferencia total del proyecto.

22.2. Preparación del emisor

1. congelar o identificar working copy;
2. actualizar documentación y estado;
3. ejecutar pruebas mínimas;
4. enumerar trabajo no commitado;
5. actualizar defectos/riesgos;
6. preparar accesos por canal seguro;
7. verificar backups;
8. redactar handoff con próximo paso observable.

22.3. Verificación del receptor

El receptor debe poder, sin asistencia operativa del emisor:

- localizar la baseline candidata o final correcta;
- distinguir documentos vigentes, candidatos e históricos;

- confirmar acceso, rama, `PROJECT_SOURCE_COMMIT` y working copy;
- identificar Unity, paquetes y Build Profile;
- abrir Unity o ejecutar el modo reducido documentado;
- ejecutar el test/build acordado;
- explicar sprint, gate y próxima acción;
- localizar riesgos, defectos, backups y rollback;
- verificar el paquete mediante hashes;
- redactar un plan de primer día.

La necesidad de instrucciones tácitas, rutas privadas o credenciales del propietario es un fallo de handoff y debe registrarse como finding/riesgo.

22.4. Handoff de una sesión

Debe ser breve pero contener:

- objetivo;
- qué cambió;
- archivos/escenas;
- validación;
- estado de working copy;
- problema abierto;
- siguiente paso exacto.

22.5. Handoff total

Añadir:

- inventario de cuentas, contratos y proveedores;
- propiedad intelectual y licencias;
- finanzas/costes y renovaciones;
- datos y obligaciones de privacidad;
- repositorio, commits/tags y builds;
- baseline documental, manifests y checksums;
- backups, RTO/RPO y pruebas de restauración;
- riesgos aceptados y pendientes;
- comunidad, Steam, marketing y soporte;
- acta de aceptación.

El handoff total solo puede cerrarse después de ejecutar **RB-17** o un ejercicio equivalente y registrar su evidencia en 33/31. La firma sin prueba de recepción no demuestra continuidad. *** # 23. Offboarding y transferencia de colaboradores

23.1. Salida planificada

- completar o devolver trabajo;
- entregar fuentes y `.meta`;
- eliminar secretos locales;

- transferir conocimiento;
- actualizar documentación;
- revocar accesos;
- confirmar propiedad/licencias;
- archivar comunicación contractual.

23.2. Salida urgente

- revocar primero;
- preservar logs/evidencia;
- rotar credenciales compartidas;
- inventariar ramas, forks y assets;
- recuperar dispositivos/cuentas;
- evaluar fuga o pérdida;
- reconstruir handoff desde repositorio y registros.

23.3. Forks y copias

Definir por contrato y política:

- si se permiten;
 - dónde se alojan;
 - cómo se eliminan al terminar;
 - qué datos/assets no pueden copiarse;
 - cómo se verifica la devolución.
-

24. Colaboración, revisión y entrega entre disciplinas

24.1. Contrato de colaboración

Antes de asignar trabajo a una persona interna o externa debe existir un contrato operativo, aunque el contrato jurídico se gestione por separado. Debe definir:

- objetivo y resultado entregable;
- archivos, escenas, sistemas y documentos que puede modificar;
- archivos expresamente excluidos;
- rama o mecanismo de entrega;
- formato de fuentes y derivados;
- naming, unidades, escalas y convenciones;
- licencia y cesión de derechos;
- confidencialidad;
- revisión y número de rondas;
- criterios de aceptación;
- fecha, dependencias y riesgo;

- procedimiento de devolución y borrado de copias.

No se debe enviar un ZIP completo del proyecto cuando un paquete mínimo sea suficiente. La reducción de acceso disminuye riesgo de filtración, conflicto y dependencia.

24.2. Brief técnico y creativo

Un brief útil contiene dos capas:

1. intención: fantasía, función, tono, prioridad y referencia visual o de experiencia;
2. contrato técnico: formato, dimensiones, pivote, materiales, LOD, colliders, naming, presupuesto, plataforma, accesibilidad, localización y destino de integración.

Una referencia visual no reemplaza el contrato técnico. Un contrato técnico sin intención puede producir un asset correcto pero inadecuado para el juego.

24.3. Entrega de ingeniería

El colaborador de ingeniería debe entregar:

- rama/commit o patch identificable;
- descripción de arquitectura y decisiones;
- archivos cambiados;
- tests nuevos/modificados;
- resultados de compilación y pruebas;
- migración y compatibilidad;
- riesgos, limitaciones y deuda;
- instrucciones de integración y rollback;
- impacto documental.

No se acepta una carpeta de scripts sin historia, contexto o instrucciones cuando el cambio afecta a sistemas existentes.

24.4. Entrega de arte 3D

Debe incluir, según aplique:

- fuente editable;
- FBX o formato de intercambio;
- texturas fuente y exportadas;
- materiales previstos;
- escala y orientación;
- pivote;
- UV y texel density;
- LOD;
- colliders propuestos;

- capturas de revisión;
- licencia/provenance;
- notas de importación;
- límites o incompatibilidades.

La integración en Unity se realiza en una rama o área controlada y se revisa con el Art Bible, rendimiento y escena objetivo.

24.5. Entrega de audio

Debe incluir master, derivados de implementación, sample rate/bit depth, loop points, loudness, licencia, autor, restricciones, categoría de mixer y uso previsto. La revisión incluye clipping, transiciones, mezcla, repetición, memoria y equivalentes accesibles de señales relevantes.

24.6. Entrega de UI/UX

Debe incluir estados, navegación, foco, error, loading, vacío, disabled, teclado/mando, escalado, localización, contraste, jerarquía y comportamiento responsive. Un mockup estático no basta para aceptar un flujo.

24.7. Revisión por pares

La revisión se centra en el resultado y el contrato, no en preferencias personales. El revisor clasifica comentarios:

- bloqueante contractual;
- defecto funcional;
- riesgo;
- mejora recomendada;
- pregunta;
- preferencia no normativa.

Cada comentario debe explicar impacto y criterio. La persona autora responde con corrección, justificación o propuesta de decisión.

24.8. Integración

Antes de integrar:

1. comprobar origen y permisos;
2. aislar cambios;
3. revisar diff y assets;
4. compilar/importar;
5. ejecutar pruebas del área;
6. probar escena y build cuando corresponda;
7. actualizar catálogos/licencias;
8. registrar decisión y evidencia;

9. borrar paquetes temporales solo después de verificar backup.
-

25. Registros operativos, nomenclatura y custodia

25.1. Identificadores

Usar IDs estables por tipo:

- CHG- para cambios;
- ADR- para decisiones arquitectónicas;
- BUG- o esquema definido en QA para defectos;
- R- para riesgos;
- TA- para acciones de tratamiento;
- INC- para incidentes;
- EV- para paquetes de evidencia;
- BLD- para builds;
- DOC- para acciones documentales;
- OPS- para work packages operativos.

No reutilizar un ID cerrado para otro asunto.

25.2. Fechas y zona horaria

Registrar fechas ISO YYYY-MM-DD y hora con zona cuando una secuencia o incidente dependa del tiempo. Para coordinación ordinaria del proyecto se usa Europe/Madrid, sin borrar la hora original de servicios externos.

25.3. Rutas y nombres

- evitar nombres ambiguos como `final`, `final2`, `new` o `backup_latest`;
- usar versión, fecha o build ID según el artefacto;
- mantener nombres compatibles con tooling y empaquetado;
- no incluir secretos o datos personales en nombres;
- conservar relación entre fuente, derivado y artefacto aprobado.

25.4. Evidencia y retención

Clasificar la evidencia como:

- temporal de diagnóstico;
- necesaria para cierre de tarea;
- necesaria para hito/release;
- legal/seguridad;
- histórica.

La evidencia de release, licencia, incidente o aceptación no se elimina con la misma cadencia que logs rutinarios. 25, 26, 27 y 33 definen necesidades específicas.

25.5. Ubicaciones prohibidas como única copia

- escritorio local;
- carpeta Downloads;
- carpeta temporal de Unity;
- chat;
- correo de una sola persona;
- dispositivo externo no inventariado;
- build folder sobrescribible;
- nube personal no controlada.

25.6. Minutas y decisiones

Una reunión solo produce autoridad cuando sus decisiones se registran. La minuta debe contener asistentes, hechos, decisiones, acciones, owners, fechas y documentos impactados. La grabación no sustituye el registro estructurado.

25.7. Estado de trabajo

Usar estados inequívocos: NOT STARTED, IN PROGRESS, BLOCKED, IN REVIEW, DONE, CANCELLED. DONE significa DoD cumplida, no que alguien dejó de trabajar.

26. Paquete de transferencia integral del proyecto

26.1. Nombre, ubicación y estado del paquete

La estructura canónica final será:

```
Cartridge_And_Cloud_Documentation_<BASELINE_VERSION>/
├─ README_FIRST.md
├─ VERSION.txt
├─ PACKAGE_MANIFEST.md
├─ CHECKSUMS_SHA256.txt
├─ DOCUMENTATION_VALIDATION_REPORT.md
├─ BASELINE_CHANGELOG.md
├─ RESTORE_AND_VERIFY.md
├─ Current_Baseline/
│   └─ 00_Enfoque_y_Alcance.md
│   └─ ...
│   └─ 34_Final_Handoff_and_Project_Operations_Manual.md
├─ Historical_Baselines/
```

|— Evidence/
|— Integrity/

Estado en esta versión:

Campo	Valor
BASELINE_VERSION	PENDING – decidir mediante control de cambios
ubicación final	PENDING – registrar ruta/repositorio controlado después del freeze
PROJECT_SOURCE_COMMIT	PENDING VERIFICATION
DOCUMENTATION_TAG	PENDING APPROVAL
ZIP final	NOT GENERATED
checksum externo del ZIP	NOT GENERATED
restore test	NOT RUN
independent handoff	NOT RUN

Las carpetas `Documentation_RC1` y `Documentacion_Tras_RC1` son fuentes de trabajo y genealogía; no deben publicarse como `Current_Baseline`.

26.2. Componentes obligatorios

Una transferencia total debe reunir:

1. repositorio con historia, ramas y tags necesarios;
2. working copy o stash identificado si existe trabajo no integrado;
3. baseline documental completa y sus fuentes;
4. manifests, checksums e informe de validación;
5. builds relevantes, símbolos y logs según política;
6. fuentes de arte/audio y derechos;
7. catálogos, datos y saves de prueba;
8. cuentas, roles, MFA y recuperación mediante mecanismo seguro;
9. contratos, facturas, licencias y notices;
10. roadmap, backlog, defectos, riesgos y decisiones;
11. comunidad, marketing, Steam y proveedores si están activos;
12. backups y resultado de restauración;
13. inventario de hardware, dominios y servicios;
14. acta de entrega y aceptación.

26.3. Inventario de accesos

El inventario no contiene contraseñas en texto. Contiene:

- servicio;
- propósito;

- owner legal;
- administrador primario y sustituto;
- correo de recuperación;
- MFA habilitado;
- ubicación del secreto en gestor;
- aplicaciones/tokens asociados;
- coste/renovación;
- procedimiento de revocación;
- último test de acceso.

26.4. Inventario de obligaciones

Incluir:

- pagos y renovaciones;
- compromisos con contractors;
- licencias con atribución;
- plazos de plataforma;
- comunicaciones prometidas;
- datos o solicitudes pendientes;
- riesgos aceptados con vencimiento;
- obligaciones de confidencialidad.

26.5. Prueba de recepción independiente

El receptor debe ejecutar **RB-17** o un ejercicio equivalente de “día cero”:

1. activar y registrar el alcance de autoridad del sustituto;
2. localizar **34**, **14**, **15** y el último paquete verificado;
3. verificar manifest y hashes;
4. recuperar accesos de mínimo privilegio sin ayuda del propietario;
5. localizar commit/tag, Unity, paquetes y build aceptada;
6. revisar riesgos, gates, defectos y secuencia documental;
7. abrir el proyecto o ejecutar el modo reducido documentado;
8. redactar un plan de primer día y decisiones bloqueadas;
9. completar checklist con tiempos y evidencias;
10. corregir gaps y programar repetición.

La transferencia no se considera terminada hasta documentar el resultado y obtener aceptación del receptor y del owner competente.

26.6. Transferencia parcial

Cuando se transfiere solo una disciplina, se entrega el subconjunto mínimo, pero también las dependencias y restricciones que puedan bloquearla. Un artista no necesita secretos de Steam, pero sí Art Bible, catálogo, licencias, budgets, escena de integración y proceso de revisión.

26.7. Custodia posterior

Después de la aceptación:

- conservar copia de entrega según contrato;
- revocar accesos del emisor cuando corresponda;
- registrar nuevo owner;
- actualizar BIA y dependencias;
- cambiar contactos y facturación;
- comprobar que backups y alertas usan cuentas vigentes;
- conservar el checksum y acta de la copia exacta recibida. *** # 27. Comunicación, decisiones y escalado

27.1. Canales por función

Separar, cuando existan herramientas adecuadas:

- decisiones y governance;
- tareas y defectos;
- conversación rápida;
- archivos/evidencia;
- incidentes;
- secretos.

No enviar secretos por el canal de conversación general ni usar comentarios de commits para decisiones comerciales sensibles.

27.2. Mensaje operativo eficaz

Debe contener:

- contexto mínimo;
- hecho observado;
- impacto;
- acción solicitada o decisión requerida;
- fecha límite real;
- enlaces/IDs de evidencia.

Evitar mensajes como “no funciona” o “está terminado” sin build, escenario y criterio.

27.3. Escalado de bloqueantes

Escalar cuando:

- una dependencia bloquea la ruta crítica más allá del umbral;
- aparece S0/S1;
- falta licencia o acceso crítico;
- existe riesgo de pérdida de datos;

- una decisión afecta alcance, schema, arquitectura o release;
- se sospecha incidente de seguridad/privacidad;
- el presupuesto o calendario supera tolerancia.

El escalado incluye opciones y recomendación, no solo el problema.

27.4. Decisiones reversibles e irreversibles

- reversibles: experimento visual, parámetro de balance, herramienta local; pueden decidirse cerca del trabajo con límites;
- costosas: schema, IDs, paquetes, arquitectura, escena de arranque, contrato, naming público, Steam; requieren mayor evidencia y autoridad.

27.5. Silencio y ausencia

Si una decisión no llega:

1. registrar bloqueo;
2. aplicar opción segura y reversible cuando exista;
3. no inventar aprobación;
4. conservar trabajo preparatorio separado;
5. revisar prioridad.

27.6. Comunicación externa

Solo personas autorizadas comunican fechas, precio, plataforma, features, incidentes o datos. Los mensajes deben derivar de la casa de mensajes y matriz de claims de 28.

28. Indicadores de salud operativa

28.1. Salud de repositorio

- main compila;
- working copies no acumulan cambios desconocidos;
- ramas envejecidas revisadas;
- LFS/objetos accesibles;
- backups verificados;
- secretos ausentes;
- tags/releases auditables.

28.2. Salud de producción

- WIP limitado;
- próxima acción clara;

- dependencias con owner;
- bloqueantes con antigüedad visible;
- scope changes registrados;
- gates no se cierran por presión de calendario.

28.3. Salud de calidad

- suite estable;
- flaky tests identificados;
- S0/S1 con respuesta;
- builds externas periódicas;
- evidencia recuperable;
- regresiones de sistemas cerrados controladas.

28.4. Salud documental

- documentos competentes actualizados;
- hashes y manifests vigentes;
- referencias no rotas;
- estados distinguen plan/implementación/prueba;
- histórico inmutable;
- nueva persona encuentra información sin chat.

28.5. Salud de continuidad

- accesos con sustituto;
- MFA/recovery probados;
- backups recientes;
- restauración ejecutada;
- proveedores críticos inventariados;
- playbooks ejercitados;
- riesgos fuera de tolerancia disminuyen o tienen decisión.

28.6. Señales de deterioro

- “solo funciona en mi PC”;
- builds sin SHA;
- assets sin licencia;
- cambios masivos no explicados;
- tests verdes usados para ignorar aceptación manual;
- archivos `final_final`;
- documentos que contradicen código sin ticket;
- una sola persona controla todos los accesos;
- backups nunca restaurados;
- tareas cerradas sin evidencia;
- publicación basada en promesas futuras.

28.7. Revisión de salud

Producción consolida estos indicadores en la revisión semanal y eleva a riesgo cualquier deterioro sostenido. Los indicadores no sustituyen métricas especializadas, pero revelan temprano pérdida de control.

29. Work packages para institucionalizar la operación

ID	Paquete	Resultado	Evidencia de cierre
OPS-01	Verificar repositorio activo	rama, SHA, remoto y working copy confirmados	status report
OPS-02	Actualizar handoff vivo	estado, próximo paso y riesgos actuales	handoff aprobado
OPS-03	Restauración de prueba	proyecto recuperado en entorno limpio	informe y tiempos
OPS-04	Inventario de cuentas	owner, MFA, recovery y sustituto	registro seguro
OPS-05	Limpieza de secretos	ausencia de credenciales versionadas	scan y revisión
OPS-06	Cierre StoreInitial	escena autorada y conectada	evidencia S16
OPS-07	Build postintegración	candidata ejecutada fuera de Editor	build record/log
OPS-08	Cierre Sprint 16	signoff completo	closure report
OPS-09	Apertura Sprint 17	charter y aceptación congelados	brief aprobado
OPS-10	Baseline de rendimiento	escenarios/hardwarebenchmark	report
OPS-11	Revisión accesibilidad	medidos gaps y pruebas ejecutadas	matriz/evidencia
OPS-12	H6 execution	controles 31 cerrados	signoff H6
OPS-13	Regenerar 12	maestro 00-34	XLSX validado

ID	Paquete	Resultado	Evidencia de cierre
OPS-14	Regenerar 13	cambios/linaje completos	XLSX validado
OPS-15	Regenerar 14	índice 00–34	MD validado
OPS-16	Regenerar 15	guía completa	MD validado
OPS-17	Regenerar 16	auditoría final	informe sin bloqueantes
OPS-18	Actualizar 21/23	referencias y hashes	libros validados
OPS-19	Actualizar 31/32	referencias 33/34 y cierre	libros validados
OPS-20	Manifest externo	inventario y hashes finales	manifest
OPS-21	Checksums	SHA-256 de paquete	archivo checksum
OPS-22	Informe validación	estructura, enlaces, formatos	report PASS
OPS-23	ZIP inmutable	baseline final	hash y extracción PASS
OPS-24	Copia independiente	recuperación protegida	verificación
OPS-25	Onboarding drill	tercero retoma sin chat	acta de ejercicio
OPS-26	Crisis tabletop	playbook ejecutado	lessons/actions
OPS-27	Release access drill	Steam/Git/backup sustituto	evidencia controlada
OPS-28	Revisión trimestral	manual y BIA actualizados	change log

30. Checklists operativos

30.1. Antes de editar

- ☐ He leído la fuente competente.
- ☐ Sé qué criterio debe cambiar o cumplirse.
- ☐ Conozco rama, SHA y cambios locales.
- ☐ Tengo rollback.
- ☐ He identificado tests y riesgos.
- ☐ No estoy usando una afirmación histórica como verdad actual.

30.2. Antes de commit

- ☐ Diff revisado.
- ☐ Sin secretos ni archivos temporales.
- ☐ `.meta` correctos.
- ☐ Compila.
- ☐ Tests proporcionales.
- ☐ Escena/prefab reabiertos si se tocaron.
- ☐ Documentación y trazabilidad actualizadas.
- ☐ Mensaje describe resultado y validación.

30.3. Antes de build candidata

- ☐ Working copy limpia.
- ☐ SHA y versión congelados.
- ☐ Build Profile correcto.
- ☐ escenas correctas.
- ☐ tests requeridos PASS.
- ☐ defectos/excepciones revisados.
- ☐ legal, seguridad y contenido revisados.
- ☐ ruta de salida limpia.
- ☐ manifest y evidencia preparados.

30.4. Antes de cerrar sprint

- ☐ scope y DoD.
- ☐ acceptance matrix.
- ☐ QA y manual.
- ☐ build/log cuando aplica.
- ☐ defectos/deuda.
- ☐ ADR/cambios.
- ☐ documentación.
- ☐ riesgos.
- ☐ handoff.
- ☐ signoff.

30.5. Antes de transferir

- ☐ repositorio y SHA.
- ☐ working copy.
- ☐ baseline y hashes.
- ☐ accesos/MFA/recovery.
- ☐ backups/restauración.
- ☐ estado, próximos pasos y bloqueantes.
- ☐ builds/evidencia.
- ☐ riesgos y excepciones.

- ☐ contratos/licencias.
 - ☐ receptor ha reproducido el entorno.
-

31. Plantillas

31.1. Plantilla de handoff operativo

Handoff – <fecha> – <ámbito>

Identidad

- Entrega:
- Recibe:
- Rama / SHA:
- Working copy:
- Unity / paquetes:
- App / schema / baseline:

Estado

- Sprint / gate:
- Resultado terminado:
- En curso:
- No iniciado:
- Bloqueantes:

Validación

- Tests:
- Build:
- Manual / Golden Path:
- Logs / evidencia:

Riesgos y decisiones

- Riesgos:
- Excepciones:
- Decisiones pendientes:

Próximo paso observable

1. ...

Rollback / recuperación

- ...

Aceptación del receptor

- Fecha:
- Resultado de reproducción:

31.2. Plantilla de status report

```
# Project Status – <fecha>
- Baseline:
- Branch/SHA:
- Sprint/Gate:
- Completed:
- In progress:
- Blocked:
- Tests/build:
- Defects S0/S1:
- Risks outside tolerance:
- Decisions needed:
- Next milestone:
```

31.3. Plantilla de decisión

```
# Decision <ID> – <título>
- Contexto:
- Opciones:
- Decisión:
- Autoridad:
- Consecuencias:
- Riesgos:
- Documentos/código impactados:
- Tests:
- Rollback:
- Fecha/revisión:
```

31.4. Plantilla de incidente

```
# Incident <ID>
- Detectado:
- Severidad:
- Sistemas/datos:
- Timeline:
- Contención:
- Evidencia:
- Causa conocida/hipótesis:
- Recuperación:
- Comunicación:
- Acciones:
- Cierre y revisión:
```

31.5. Plantilla de build record

```
# Build Record <Build ID>
- App version / schema:
- Branch / SHA:
- Unity / packages:
- Profile / scenes / backend:
- Flags:
- Tests previos:
- Output:
- SHA-256:
- Smoke / Golden Path:
- Player.log:
- Defects/exceptions:
- Decision:
```

32. Criterios de aceptación de este manual

Esta versión se considera aceptable como **manual operativo candidato** cuando:

1. permite retomar el proyecto sin historial de chat;
2. distingue fotografía histórica, estado candidato y verdad actual;
3. integra las baselines históricas y los registros vivos anteriores;
4. describe el estado técnico, de producción y de gate sin declarar cierres inexistentes;
5. explica jerarquía 00-34 y resolución de contradicciones;
6. define onboarding, setup, Git, Unity, arquitectura y ciclo de trabajo;
7. cubre QA, builds, documentación, assets, accesibilidad, rendimiento, legal, privacidad y seguridad;
8. integra los 192 riesgos, continuidad, backups, incidentes y recuperación del documento 33;
9. define handoff independiente, offboarding, rutinas y playbooks;
10. incluye checklists y plantillas reutilizables;
11. identifica los documentos ya regenerados y la secuencia exacta hacia 32/16;
12. define paquete, hashes, restauración y tratamiento de autorreferencias;
13. no inventa commit, tag, ubicación final, build ni evidencia;
14. conserva fuentes históricas y remite al inventario exhaustivo de 32.

Para considerarlo parte de la **baseline final**, además deben completarse:

- auditoría 16 final;
- manifiesto 32 definitivo;
- actualización por findings aplicables;
- manifest/checksums/report externos;

- ZIP immutable;
- restauración limpia PASS;
- handoff independiente PASS;
- commit del proyecto y tag documental reales.

Estado del documento: OPERATIONAL CANDIDATE COMPLETE / FINAL BASELINE AND INDEPENDENT HANDOFF PENDING.

33. Registro de la regeneración dirigida de este manual

Change ID	Cambio	Motivo	Estado
CHG34-001	versión y estado cambiados a 1.1-RC1	distinguir manual candidato de baseline final	aplicado
CHG34-002	incorporados estados reales de 12-15, 21, 23, 31 y 33	eliminar referencias RC1 obsoletas	aplicado
CHG34-003	34 deja de describirse como cierre final automático	evitar falso HANDOFF READY	aplicado
CHG34-004	secuencia actualizada a 32 candidata → 16 candidata → correcciones → finales	alinear Binder, Guía, 31 y 33	aplicado
CHG34-005	definidos paquete canónico y artefactos externos	preparar freeze verificable	aplicado
CHG34-006	añadido tratamiento de SHA/commit autorreferencial	impedir ciclos de integridad imposibles	aplicado
CHG34-007	restauración limpia y handoff independiente vinculados a RB-16/RB-17	exigir evidencia operativa	aplicado

Change ID	Cambio	Motivo	Estado
CHG34-008	cifras de riesgos/continuidad actualizadas	sincronizar con 33 regenerado	aplicado
CHG34-009	inventario 00-34 y hashes candidatos actualizados	sincronizar fuentes actuales	aplicado
CHG34-010	numeración interna de checklists/plantillas corregida	mejorar coherencia estructural	aplicado

La pasada final de este documento queda limitada a cambios reales derivados de la auditoría, nombre/versión del paquete, `PROJECT_SOURCE_COMMIT`, `DOCUMENTATION_TAG`, ubicación final y resultados de restore/handoff. No debe reescribirse sin motivo, porque cualquier modificación invalida su hash y los artefactos dependientes. *** # Anexo A. Inventario operativo de documentos 00-34

La tabla refleja la candidata utilizada para esta regeneración. El manifiesto 32 candidato debe reproducir estos estados y hashes; cualquier cambio posterior los invalida.

ID	Documento	Autoridad / uso operativo	Estado actual	Bytes	SHA-256 externo
00	00_Enfoque_y_Alianza	Visión, pilares y límites de alcance	PRESENTE / VIGENTE PARA SU ÁMBITO	127401	63dd6f0f1b9c2c80be2aec55718d18d9d031ce
01	01_Game_Design_Document	Discurso de juego y experiencia sistémica	PRESENTE / VIGENTE PARA SU ÁMBITO	132822	a9cd9e3203abc5269c25c59dc7f626b0771df
02	02_Vertical_Align_Specification	Alcance verificable, Golden Path y H6	PRESENTE / VIGENTE PARA SU ÁMBITO	64531	75893f130116e88a08b8da5590dd81876a234f

ID	Documento	Autoridad / uso operativo	Estado actual	Bytes	SHA-256 externo
03	03_Technical_Design_Document	Diagramas de runtime y contratos técnicos	PRESENTE / VIGENTE PARA SU ÁMBITO	101994	66264703c5ff4959d03093690e9845a98ec0e5
04	04_Modelo_de_datos	Diagrama de persistencia e invariantes	PRESENTE / VIGENTE PARA SU ÁMBITO	102948	d851a72b55742c2c704cc7c002929bc5894e73
05	05_UX_Flow	Diagrama de flujos, navegación, estados y mensajes	PRESENTE / VIGENTE PARA SU ÁMBITO	56805	e08da5ff67fb073a8b950babe325ae58ddb2e3
06	06_Production_Roadmap	Diagrama de sprints, gates y secuenciación	PRESENTE / VIGENTE PARA SU ÁMBITO	61564	d00b156c0ad1c66069278119c55f76c738a577
07	07_QA_Testing_Strategy	Diagrama de estrategia, severidades y proceso QA	PRESENTE / VIGENTE PARA SU ÁMBITO	79435	bc9c05a3737e345546ef16e5390e034295ef35
08	08_QA_Testing_Guide	Diagrama de prueba y cobertura	PRESENTE / VIGENTE PARA SU ÁMBITO	185032	233e57f01d2468fe13c136c8fcd3ee75b39bd3
09	09_CSharp_Coding_Standards	Diagrama de estándares y calidad C#	PRESENTE / VIGENTE PARA SU ÁMBITO	95321	3b734d6cd49f31c09c10bb4941625e143f6c63

ID	Documento	Autoridad / uso operativo	Estado actual	Bytes	SHA-256 externo
10	10_Unity_Project_Setup	reproducible de Unity y proyecto	PRESENTE / VIGENTE PARA SU ÁMBITO	78408	31430bb0544edd9577323aa0009d6ca3df8b91
11	11_Build_y_Versioning	builds, ramas e integridad	PRESENTE / VIGENTE PARA SU ÁMBITO	85324	38a16a0f97505c0b405509dd3c7e3d1b50e77b
12	12_Excel_Maestro	Identificación y seguimiento maestro	RECIBIDO TRAS RC1 / REVALIDACIÓN FINAL PENDIENTE	484932	3d70e97326868389dac977934e6000285c22b3
13	13_Trazabilidad	Traza y documentación	RECIBIDO TRAS RC1 / REVALIDACIÓN FINAL PENDIENTE	577554	97cf106d7707de9e6dcadfa2ec9374676b0ea5
14	14_Project_Finder	Índice de autoridad y navegación	RECIBIDO TRAS RC1 / REVALIDACIÓN FINAL PENDIENTE	95577	d0e8c2b0bf78a414d00c290d8ab8e4b2742e80

ID	Documento	Autoridad / uso operativo	Estado actual	Bytes	SHA-256 externo
15	15_Guia_Maestros	15_Guia_Maestros.md integrado de la documentación	REGENERADO TRAS RC1 / REVALIDACIÓN FINAL PENDIENTE	149595	70aa63feb69785d3b2e8b044c73b9c5f075829
16	16_Auditoria_Chequeo de contradicciones y findings	16_Auditoria_Chequeo de contradicciones y findings.md	REVALIDACIÓN AUDITORÍA CANDIDATA Y FINAL PENDIENTES	87439	4a62376b8aee623be40957ae0511d2dcab2919
17	17_Art_Bible	17_Art_Bible.md Dirección visual y producción artística	PRESENTE / VIGENTE PARA SU ÁMBITO	133913	095c6cc42223a972c7faa68435595fc7c22f83
18	18_Audio_Bible	18_Audio_Bible.md Dirección sonora y pipeline de audio	PRESENTE / VIGENTE PARA SU ÁMBITO	144525	c2eabe93a7c26f70e1d4d77fb0ddcaa97e2cd0
19	19_UI_Style_System	19_UI_Style_System.md Dirección visual y componentes UI	PRESENTE / VIGENTE PARA SU ÁMBITO	149800	b34ca4db1eba7a536466b6a9a105d8d92396a0
20	20_Economy_and_Balance	20_Economy_and_Balance.md End Balance balance y telemetría de diseño	PRESENTE / VIGENTE PARA SU ÁMBITO	154547	908c70772ed35eecd05c3036c6901523a4ca1

ID	Documento	Autoridad / uso operativo	Estado actual	Bytes	SHA-256 externo
21	21_Initial_Content_Initial_Coverage	Contenta- inicial, IDs y cobertura	REGENERADO TRAS RC1 / REVALI- DACIÓN EN 16 PEN- DIENTE	186599	2f98e2b7070e5a1f70d4feee03a7d154a2dcde
22	22_Localization_Handling	idiomas y pipeline	PRESENTE / VIGEN- TE PARA SU ÁMBITO	157323	184a53b2813bc1327a6913e6e88ea4c81142da
23	23_Legal_Credentia- licencias y prove- nance	Legal and Licenses	REGENERADO TRAS RC1 / REVALI- DACIÓN EN 16 PEN- DIENTE	189706	df975425a8b8c1cfd9375179893a68422c5693
24	24_Steam_Publicing_Risk	store, builds y lanza- miento	PRESENTE / VIGEN- TE PARA SU ÁMBITO	171037	f087e24d3a18f8c4a6651728d5f70d1df8040a
25	25_Post_Launch_Support_Live_Operations	actualiza- ciones y live ope- rations	PRESENTE / VIGEN- TE PARA SU ÁMBITO	27218	158e1e43514c5d5d6c365f6cd1a73929f54c4a
26	26_Privacy_Data_and_Teleme- tría, consenti- miento y retención	Data and Teleme- tría, consenti- miento y retención	PRESENTE / VIGEN- TE PARA SU ÁMBITO	321182	284abfe537793c2ec8b0e5cbac484cd9395567

ID	Documento	Autoridad / uso operativo	Estado actual	Bytes	SHA-256 externo
27	27_Security_Sand Incident Response Plan.md	Sancti Incident Response Plan, vulnerabilidades e incidentes	PRESENTE / VIGENTE PARA SU ÁMBITO	22182	f925bfbf3df2000323c12a4486480d06c8ab83
28	28_Marketing_Marketing Communication, community and campaign	Marketing Communication, comunidad y campaña	PRESENTE / VIGENTE PARA SU ÁMBITO	87510	28dd332855126558b3da6019f2e546b1e8ab70
29	29_Accessibility_Inclusion, QA and claims	Accessibility, Inclusion, QA y claims	PRESENTE / VIGENTE PARA SU ÁMBITO	405005	678ceabb509062cd93c7690315397b1cc62040
30	30_Performance_Benchmarks, profiling and optimization	Performance Benchmarks, profiling y optimización	PRESENTE / VIGENTE PARA SU ÁMBITO	109482	18f53eaa61734ad735c02c8b7c70ed37f5f0e8
31	31_H6_and_Release_Ready, Checklists	H6, Release y gates documentales	PRESENTE / 280 H6 Y 223 RELEAS NOT RUN	19011	c61a903e5a6d5944b60c4ffde403f6184bc960
32	32_Documentation_Base, hashes, authority and substitutions	Documentation Base, hashes, autoridad y sustituciones	PRESENTE / REGISTRATION CANDIDATE PEN-DIENTE	197250	05ef1aa53febb8015b1eb40be8f43a4f6cee51

ID	Documento	Autoridad / uso operativo	Estado actual	Bytes	SHA-256 externo
33	33_Project_Risks_Register	Risks Register globales, backups y recuperación	RECEIVED, IN PROGRESS, NOT VERIFIED	777	2bde968afc77c8a2ad3c8b28c06fcb0542d1ae
34	34_Final_Handoff_and_Project_Operation_Manual	Handoff and Project Operation Manual	EN ESTA PASADA FINAL LIMITADA PENDIENTE	AFTER EXPORT	EXTERNAL EXPORT

Anexo B. Ledger histórico de handoffs, estados y cierres

Este ledger reúne los documentos históricos directamente relacionados con retoma, estado, cierre y operación. Las copias idénticas se conservan porque su ruta demuestra el contexto de publicación. El inventario completo de los 745 archivos históricos se mantiene en 32_Documentation_Baseline_Manifest.xlsx. **Entradas operativas listadas:** 84. | Ruta histórica | Bytes | SHA-256 | Clasificación | |—|—:|—|—| | Documentation/00_Official_Baseline/v0.3/README.en.md | 2005 | 161af250279dce492d52047789a719f117c739cf1435c1a8d6e1660c675a3858 | Baseline publicada / histórico inmutable | | Documentation/00_Official_Baseline/v0.3/README.md | 2026 | fcc1f961b49d3f9e444a10bd58b690d6a16f5973612c3614beadb2a22b9720bb | Baseline publicada / histórico inmutable | | Documentation/00_Official_Baseline/v0.4/06_Development_Records | 466 | f789933c54973d329e078dd5a561b43f174333653a36aca00ce5254daaac0aab | Baseline publicada / histórico inmutable | | Documentation/00_Official_Baseline/v0.4/06_Development_Records | 1233 | 9f9d70fae2a2197c724f273e1279b1589c19641519e2795bda3ad32e954b5556 | Baseline publicada / histórico inmutable | | Documentation/00_Official_Baseline/v0.4/06_Development_Records | 856 | 40304282ce444aae9a485d1f9cbb27b6157460aef53ebee86d2c1e8c5aa7999a | Baseline publicada / histórico inmutable | | Documentation/00_Official_Baseline/v0.4/06_Development_Records

| 758 | 89e8091242b513ea0e04589d73cf525f6a6f17b3c4d4d7da2b820737197083f8 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.4/06_Development_Records
| 1238 | 0fed73bbd55bee471c7c2858bd09fcf946b308aab364d00455595c10a2b2e877 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.4/README.en.md
| 298 | d428feb2eda48918d7cbacb78b44b8fff9bb6cf01f9ab3b096444ad37814810a |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.4/README.md
| 692 | c2e7a9edfe7df62aeeaf2092f66da7df4ed6b69f6a54527c92ca69c755fc327e |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 2275 | 9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 610 | 11842fa86ab349e6971855eb83766245e017dc00583dc7f50018bfbcb40fc8c03 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 320 | a9c699e34ceae21c01f791e8e9fc8fd817ad2ff576ba5a14b42a46b447e22663 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 1684 | 3cef15c3e2f9c35c725c4a00cc366321fef51e35e9b4802713e8527dbe7ccf35 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 1841 | 2b8dccc6fe9f2b2d9adff4bb17403626e70f533157a89d558c15549d2aaef69e |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 1868 | 5af2cc8138ca5ef7d49e84c24974f5d1c5939f84dc9f356d6563c417067c3ef7 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 1900 | 5be13768cf7133b2e1ccaf90c13f0fbac1186bb46125b2512559a0943d2d3bf9 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 1914 | 45e1fcd518bdb144ab357425bf1e3e37b14f33f61900b34942835d4f3a568235 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 2034 | 48d29b9fe0b1c87abd372919283cf390565a14d55092d5793e178cf431b1795a |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 2339 | aa5c70956ee39b9b3d3ac6bfa075cc267f0544f776de636fb49e5abe03d8998d |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 2350 | caa820477ab6fee79beb3b54d913463b98021e84b6e83f8cfe9f63849b577039 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 2280 | 2a2ca166756565fd85ea0bcbdd05a5025bc352562fb251a42050ab35e9809b241 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/06_Development_Records
| 2620 | 8065ffa0649ba8c67d0c754cc551d1db3f721aeadc071e9f8317a9c97264e11c |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/CURRENT_PROJECT_HANDOF
| 2275 | 9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/README.en.md
| 143 | 4dc27a1de6f10067695431be262b50332b621f9cf09891d1b02e136ef1d634f6 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.5/README.md
| 780 | 083a4ce23e27a2551535bff94edd4af1b95bb0d025d11b859bd8ba1b32b75784 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.6/06_Development_Records
| 1241 | 7a297d84a36ecdd9e4296a631fc5acbff882218d125e63312850e8f28b46cd87 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.6/06_Development_Records
| 896 | e0cca56b839dc2071363b310c06543178dbf978622990fc7f4be301e077fc754 |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.6/06_Development_Records
| 3630 | 17c317dacaec6438f7a1180b6b2811184a0d995fb41964ed2fe9ab46e1d35aab |
Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.6/06_Development_Records

| 665 | bc336c8bf852d942d1b0289cebfdde9db889088990fcc6151df5db559e94d1a9 |
 Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.6/06_Development_Records
 | 239 | fb2c1a557e3b6cd765497315fa554eacc048c2fd21a2be9c7f5f28895a189ac1 |
 Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.6/CURRENT_PROJECT_HANDOFF
 | 1241 | 7a297d84a36ecdd9e4296a631fc5acbff882218d125e63312850e8f28b46cd87 |
 Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.6/README.en.md
 | 124 | 44459d4d622f5c66ee3848ff0e1a0a0fa8879f97af60e3f619092cb84cb1fd7e |
 Baseline publicada / histórico inmutable || Documentation/00_Official_Baseline/v0.6/README.md
 | 471 | ac0e90bdca7aa7a46b4affcd7ff9b755586339a228473d3b4eb1645eeca82a79 |
 Baseline publicada / histórico inmutable || Documentation/10_Development_Records/ADR/ADR-
 0008-Sprint-0-Foundation-Closure-Policy.md | 1341 | 76b9266ef045590959eef0187fff7fa2c4dcbfa7f6cf17315987
 | Cierre / evidencia || Documentation/10_Development_Records/Handoff/CURRENT_PROJECT_HANDOFF.md
 | 1168 | ea48666ee43dbf978bef364ee645cd3ff32f31e4e6dfbd96b9edfab316546028 |
 Handoff || Documentation/10_Development_Records/Production_Tracking/README_Production_Tracking.md
 | 452 | 5c4df824d89d85e6f8885cd5285412412ef3e3f943e85f6ffe1f4b3cbd3f73e0 |
 Orientación / gobierno || Documentation/10_Development_Records/Sprint_01/Governance/Sprint_01_Current_St
 | 1684 | 3cef15c3e2f9c35c725c4a00cc366321fef51e35e9b4802713e8527dbe7ccf35 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_01/Sprint_01_Closure_Record.md
 | 1841 | 2b8dccc6fe9f2b2d9adff4bb17403626e70f533157a89d558c15549d2aaef69e |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_02/Governance/Sprint_02_Current_Status.
 | 1868 | 5af2cc8138ca5ef7d49e84c24974f5d1c5939f84dc9f356d6563c417067c3ef7 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_02/Sprint_02_Closure_Record.md
 | 1900 | 5be13768cf7133b2e1ccaf90c13f0fbac1186bb46125b2512559a0943d2d3bf9 |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_03/Governance/Sprint_03_Current_Status.
 | 1914 | 45e1fcd518bdb144ab357425bf1e3e37b14f33f61900b34942835d4f3a568235 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_03/Sprint_03_Closure_Report.md
 | 2034 | 48d29b9fe0b1c87abd372919283cf390565a14d55092d5793e178cf431b1795a |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_04/Governance/Sprint_04_Current_Status.
 | 2339 | aa5c70956ee39b9b3d3ac6bfa075cc267f0544f776de636fb49e5abe03d8998d |
 Estado / apertura || Documentation/10_Development_Records/Sprint_04/Sprint_04_Closure_Report.md
 | 2350 | caa820477ab6fee79beb3b54d913463b98021e84b6e83f8cfe9f63849b577039 |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_05/Governance/Sprint_05_Current_Status.
 | 2280 | 2a2ca166756565fd85ea0bcbd05a5025bc352562fb251a42050ab35e9809b241 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_05/Sprint_05_Closure_Report.md
 | 2620 | 8065ffa0649ba8c67d0c754cc551d1db3f721aeadc071e9f8317a9c97264e11c |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_06/Governance/Sprint_06_Current_Status.
 | 1307 | 393749de939a329f70e38cd7f6356f1f38f125c3a4a06495aed809813a55b401 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_06/Sprint_06_Closure_Report.md
 | 1395 | b0997bd963c49340833d3d0e259c747f97b7ecd5574b3da5a77a8e51a4da0f60 |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_07/Governance/Sprint_07_Current_Status.
 | 1542 | 28ea6d605bf2edcfd592207a4b99fae832913f988b2e6e14e228296669b4583 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_07/Sprint_07_Closure_Report.md
 | 1926 | 00a4116e3af46415a564444f21cb913a1f5d7fe57daa4639f8dd053cecb8ae1 |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_08/Governance/S8_Handoff_Update_Templat
 | 1297 | 752c345e77607a3d096548520f2ce082a816d2eeb0d01ea0d29ddb2bd32013a |
 Orientación / gobierno || Documentation/10_Development_Records/Sprint_08/Governance/Sprint_08_Current_St

| 1642 | ac4bd550a7410946c583949ddc50a47efd32b921ad7b4f175568bedf7dd4101b |
 Estado / apertura || Documentation/10_Development_Records/Sprint_08/Sprint_08_Closure_Report.md
 | 2446 | d6a2eee566596bde2f15c4cb2cb9fdd3ac0623bfcf6964714820d2ec1472d5f5 |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_09/Governance/S9_Handoff_Update_Template
 | 501 | e3d501b14918d549aab4d42cc2d289a8f1d83730600d1083109ea64ea409baae |
 Orientación / gobierno || Documentation/10_Development_Records/Sprint_09/Governance/Sprint_09_Current_Status
 | 669 | 1b5b8afdac50a3de23f1764f81d3ac749ad5701fa9408ab2286534bdd32cc984 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_09/Sprint_09_Closure_Report.md
 | 3172 | d683b02e85f86a4753e0e7fe4f84466c88cd702ffe6bacfb1743c21d11f61921 |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_10/Governance/S10_Handoff_Update_Template
 | 429 | e7893c392feb7b35d3e270034b36f9da758d73673adb7e3f50cecea5763003a0 |
 Orientación / gobierno || Documentation/10_Development_Records/Sprint_10/Governance/Sprint_10_Current_Status
 | 669 | 893435f0a6ec143e27717287e3702e0d7306e9f5da085f3fcedab40994cbdb4b |
 Estado / apertura || Documentation/10_Development_Records/Sprint_10/Sprint_10_Closure_Report.md
 | 3211 | e9a44655c0ad3974168a876bc31ebcf4d71c1995eea797d8e38b0c1034ebabcc |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_11/Governance/S11_Handoff_Update_Template
 | 422 | 3da63a8600ab4da7f1afe4032690763029d3551297fad0d51afbfa841687dad5 |
 Orientación / gobierno || Documentation/10_Development_Records/Sprint_11/Governance/Sprint_11_Current_Status
 | 679 | 824a6fd6c261f8cdf4c8ac7fa5b5454772248c28c63a006aff83581603b5e38c |
 Estado / apertura || Documentation/10_Development_Records/Sprint_11/Sprint_11_Closure_Report.md
 | 2545 | b4ae4ddc090cdb00af473e64cbc19ac2285cdad28e1face724b2e7ac8e0b7725 |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_12/Architecture/ADR-
 0049-ClosureReadinessSnapshot.md | 173 | 8bbcd47b5de9fe5d2fb48f877e7bc1452a95aab891c3fcae14d6b06fea819df
 | Cierre / evidencia || Documentation/10_Development_Records/Sprint_12/Governance/S12_Handoff_Update_Template
 | 361 | 683e99d27f7af84f319fad7c98230f6dec7f9b9eeddf8b4066f750b22c230bd0 |
 Orientación / gobierno || Documentation/10_Development_Records/Sprint_12/Governance/Sprint_12_Current_Status
 | 548 | abf1580da3fab4d76250d9b370d441fc5f448f23257d745ccdf27f100769e682 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_12/Sprint_12_Closure_Report.md
 | 2034 | e745ab6b920f0910b054d3c2c84b20db3ade4baef82e55a4560a3821febac57f |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_13/Governance/S13_Handoff_Update_Template
 | 431 | 7d91ce81d3567cd4f392f6d9784c7415d628eec2b65e7468ea5e1b694af743f8 |
 Orientación / gobierno || Documentation/10_Development_Records/Sprint_13/Governance/Sprint_13_Current_Status
 | 684 | 11fa2c9c81e37474d7bed83b95243be46a1a1015d81fd663df3cdaf55eea84e8 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_13/Sprint_13_Closure_Report.md
 | 3546 | a8c1ad0958a4fc002aa2f13f47a7cfcf5b00d5a0f050a078fa78e96a9027d8fa |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_14/Governance/S14_Handoff_Update_Template
 | 437 | 70ac9acc5473a29f44723f4e1499f027f5adbc772ae45635ebdb035f2374bd4f |
 Orientación / gobierno || Documentation/10_Development_Records/Sprint_14/Governance/Sprint_14_Current_Status
 | 655 | 1635e3b630c1b7596d2d2c3dbcc68a574ec1e0de2494ddb1c1a0d2c286040bdd |
 Estado / apertura || Documentation/10_Development_Records/Sprint_14/Sprint_14_Closure_Report.md
 | 3274 | 68527b99d210a091e21855b9d64de0cabcf8d650ae9772fbd7e68f259875edd |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_15/Governance/Sprint_15_Current_Status
 | 727 | 73b0bc0d6d52ab1ee422ae3768727e4084db01eddd16e79ec773cfeae71ccbb2 |
 Estado / apertura || Documentation/10_Development_Records/Sprint_15/Sprint_15_Closure_Report.md
 | 1205 | 343d252b625e600cebccec2fc9a37c825228dc7eab7052ace8ce353b5f4c859f |
 Cierre / evidencia || Documentation/10_Development_Records/Sprint_16/Phase_1/Governance/S16_P1_Current_Status

| 569 | 6494de60296d252da3368dab5a5850938c4a5c219d414f80bde328379f1c7f3b |
 Estado / apertura || Documentation/10_Development_Records/Sprints/Sprint_00/S0.1-S0.4_Closure_Record.md | 1536 | 0a89dac0e8a1b802ff1224ba09d7bebd436a03864de5ba6dca45e8ce989db78
 Cierre / evidencia || Documentation/10_Development_Records/Sprints/Sprint_00/S0.10_Sprint_Closure.md | 1233 | 9f9d70fae2a2197c724f273e1279b1589c19641519e2795bda3ad32e954b5556 |
 Cierre / evidencia || Documentation/10_Development_Records/Sprints/Sprint_00/S0.7_Closure_Record.md | 1263 | ada09fbc408488357fac2bf062e940c82eec7f02184e7bab467cd312f99efd1d |
 Cierre / evidencia || Documentation/10_Development_Records/Sprints/Sprint_00/S0.8_Closure_Record.md | 629 | a3990b9f15e0d2fb29c6c23983ff236e9753ffec20c291bbec2c72625730d1bf |
 Cierre / evidencia || Documentation/10_Development_Records/Sprints/Sprint_00/S0.9_Closure_Record.md | 1165 | 6042b8da9b246e5905d6835ad256beab1f7ec4ca4b4c82aaef4fa4dc3ffe25e5 |
 Cierre / evidencia || Documentation/10_Development_Records/Sprints/Sprint_00/Sprint_00_Closure_Record.md | 856 | 40304282ce444aae9a485d1f9cbb27b6157460aef53ebee86d2c1e8c5aa7999a |
 Cierre / evidencia || Documentation/10_Development_Records/Sprints/Sprint_00/Sprint_00_Current_Status.md | 758 | 89e8091242b513ea0e04589d73cf525f6a6f17b3c4d4d7da2b820737197083f8 |
 Estado / apertura || Documentation/10_Development_Records/Sprints/Sprint_00/Sprint_00_Foundation_Baseline.md | 1238 | 0fed73bbd55bee471c7c2858bd09fcf946b308aab364d00455595c10a2b2e877 |
 Orientación / gobierno || Documentation/README_Documentation_Structure.md | 1882 | 0b9951b7b274022526f4536d2d3da2aa7503251263a5cc2f1d162f130481ec35 |
 Orientación / gobierno |

Anexo C. Cobertura de fuentes históricas

C.1. Baseline v0.3

Se preservaron documentos de gobierno, diseño, técnica, datos, UX, arte, audio, UI, producción, QA, localización, legal, marketing, Steam, PDFs, fuentes Markdown, manifests, checksums y validación. Constituye el origen formal más antiguo del paquete suministrado.

C.2. Baseline v0.4

Añadió foundation, Sprint 0, ADR, registros de entorno, build final, cierre, matriz de impacto, manifest y validación revisados.

C.3. Baseline v0.5

Consolidó Sprints 0–5, handoff, historial, ADR, acceptance matrices, QA execution, validation checklists, build records, applied changes y trazabilidad.

C.4. Registros vivos Sprints 6–15

Conservan la evolución de Product & Inventory, Suppliers, Displays, Customers, Shopping, Checkout, Day Cycle, Economy, Save Integration y UI/UX. Sus han-

doff templates y cierres se han usado para definir la disciplina operativa de este manual.

C.5. Baseline v0.6 y Sprint 16/17

Conserva la fotografía inmediata del proyecto: 0.0.17, tests 1285 PASS, integración representativa, StoreInitial no aceptada, build postintegración pendiente y Sprint 17 restringido a estabilización.

C.6. Fuentes de trabajo

Se mantienen working documents, concept art, reportes JSON, material de authoring y archivos históricos. Su inclusión no les concede autoridad normativa, pero protege provenance y capacidad de reconstrucción.

Anexo D. Paquetes Unity observados

Paquete	Versión	Uso operativo
com.unity.ai.navigation	2.0.13	NavMesh y navegación
com.unity.ide.visualstudio	2.0.26	integración IDE
com.unity.inputsystem	1.19.0	input y UI
com.unity.localization	1.5.12	localización
com.unity.render-pipelines.universal	17.3.0	render URP
com.unity.test-framework	1.6.0	EditMode/PlayMode
com.unity.ugui	2.0.0	UI

Los módulos built-in listados en `manifest.json` también forman parte de la fotografía. Cualquier cambio requiere revisión de lockfile, compatibilidad, licencias, QA, rendimiento y rollback.

Anexo E. Glosario operativo

Término	Definición
Baseline	paquete aprobado, identificado e inmutable
Handoff	transferencia verificable de estado, conocimiento y control

Término	Definición
Working copy	archivos locales y cambios respecto al repositorio
SHA	identificador de commit o hash de contenido según contexto
Build ID	identificador único de un artefacto compilado
Gate	decisión formal basada en criterios y evidencia
Golden Path	recorrido representativo obligatorio del vertical slice
Provenance	origen, autoría, licencia y transformación de un asset
RTO	objetivo de tiempo de recuperación
RPO	objetivo de pérdida máxima de datos
MTPD	interrupción máxima tolerable
Runbook	secuencia detallada de recuperación
Playbook	respuesta coordinada a un escenario
Rollback	retorno controlado a estado anterior
Signoff	aprobación explícita por rol competente
Deuda aceptada	incumplimiento registrado con owner y revisión
Evidencia	artefacto que permite auditar una afirmación

Anexo F. Secuencia autorizada posterior a esta versión

1. Copiar esta versión de `34_Final_Handoff_and_Project_Operations_Manual.md` a la carpeta de trabajo controlada.
2. Calcular y registrar externamente su SHA-256 candidato.
3. Regenerar `32_Documentation_Baseline_Manifest.xlsx` como manifiesto candidato.
4. Verificar que 32 registra 12–15, 21, 23, 31, 33 y 34 con sus estados actuales.
5. Regenerar `16_Auditoria_Global_de_Coherencia.md` sobre el conjunto completo 00–34.
6. Clasificar findings como **BLOCKER**, **HIGH**, **MEDIUM**, **LOW** o **INFORMATIONAL**.
7. Registrar findings en 13, controles bloqueantes en 31 y riesgos nuevos/agravados en 33.
8. Corregir los documentos fuente y dependientes.
9. Recalcular hashes candidatos e invalidar los anteriores.

10. Regenerar 16 como auditoría final.
11. Regenerar 32 como manifiesto definitivo.
12. Aplicar a 34 solo la pasada final limitada que exijan findings o datos reales de paquete/Git.
13. Generar PACKAGE_MANIFEST.md.
14. Generar CHECKSUMS_SHA256.txt.
15. Generar DOCUMENTATION_VALIDATION_REPORT.md.
16. Generar VERSION.txt, README_FIRST.md, BASELINE_CHANGELOG.md y RESTORE_AND_VERIFY.md.
17. Construir el paquete desde una lista blanca en una carpeta limpia.
18. Generar el ZIP y publicar su checksum fuera del propio ZIP.
19. Extraer en un destino independiente y ejecutar RB-16.
20. Ejecutar RB-17 con un operador alternativo.
21. Registrar PROJECT_SOURCE_COMMIT, DOCUMENTATION_TAG y la ubicación final.
22. Conservar DOCUMENTATION_COMMIT_SHA externamente o en el tag anotado.
23. Cerrar G-BASE y G-DGC-RESTORE únicamente con evidencia y signoff.

Siguiente documento: 32_Documentation_Baseline_Manifest.xlsx — versión candidata.

Fin del documento 34.